

CMPE 256 – Recommender Systems

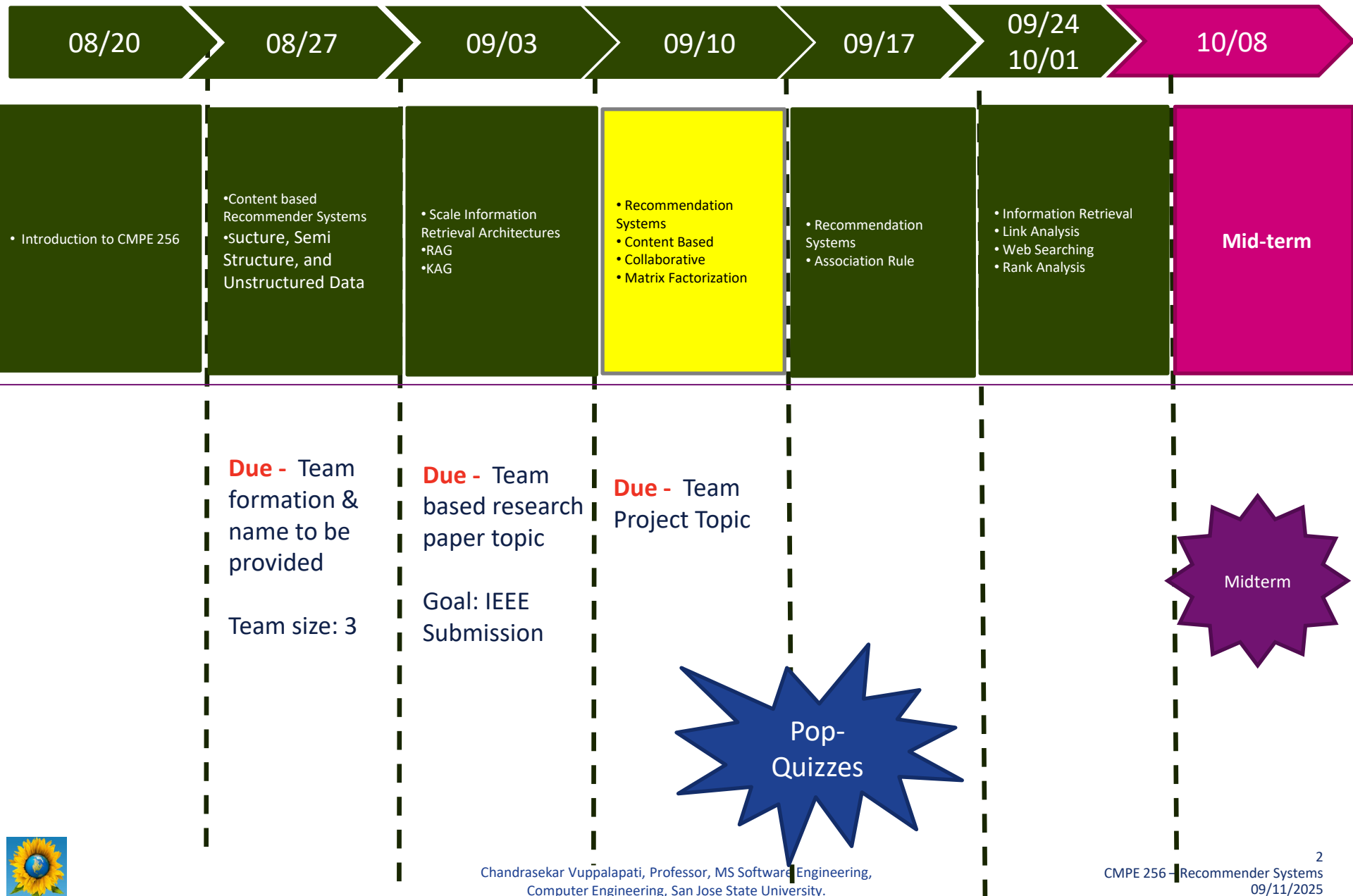


Chandrasekar Vuppalapati

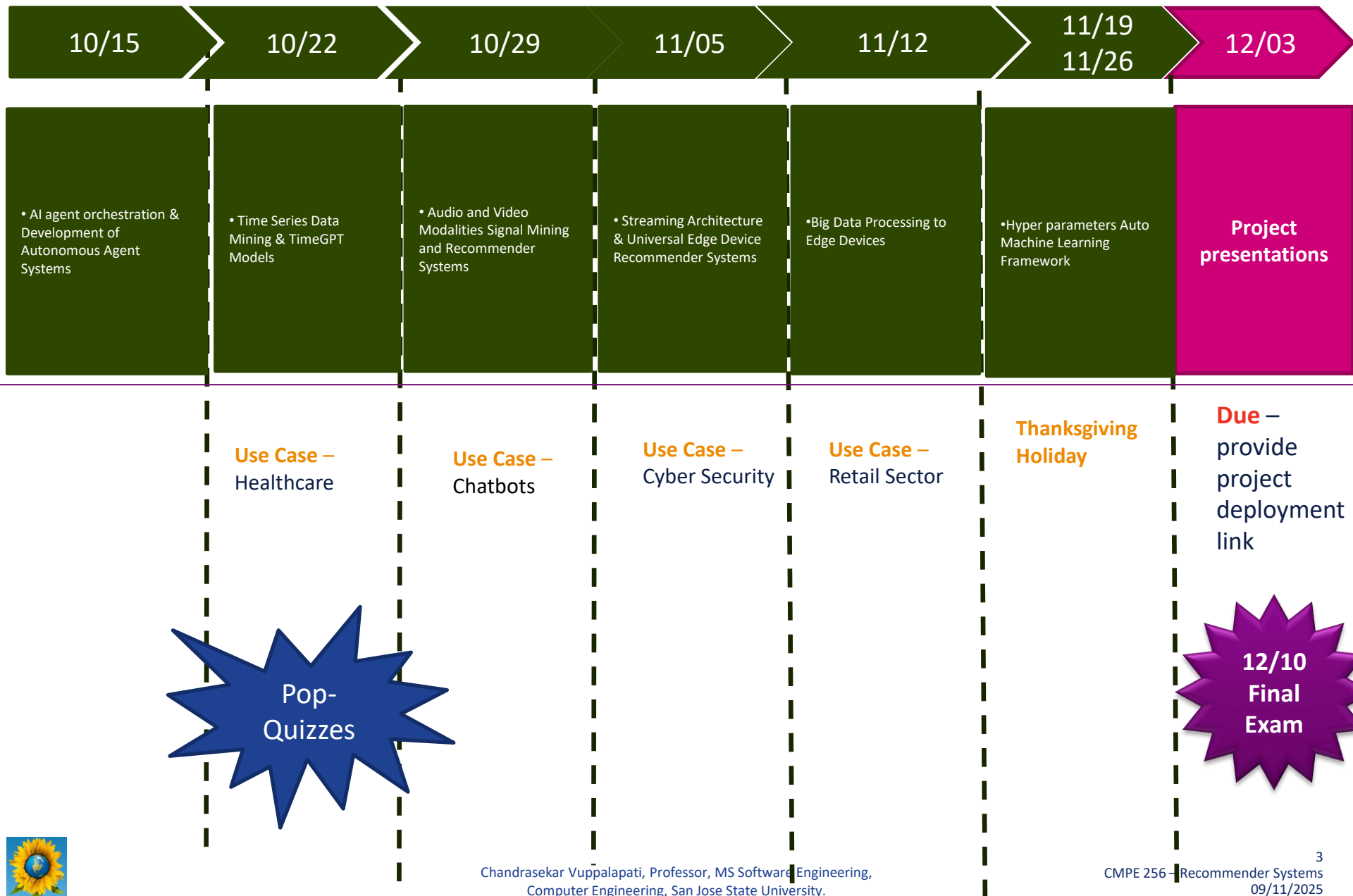
Professor, MS Software Engineering – Data Science,
Department of Computer Engineering, San Jose State
University.



Class Roadmap



Class Roadmap



Agenda

1. Information Retrieval
2. Case Study
3. Information Retrieval Models



$$\frac{\partial}{\partial \theta} \int_{\mathbb{R}_n} T(x) f(x, \theta) dx = \int_{\mathbb{R}_n} \frac{\partial}{\partial \theta} T(x) f(x, \theta) dx$$

$$\frac{\partial}{\partial a} \ln f_{a, \sigma^2}(\xi_1) = \frac{(\xi_1 - a)}{\sigma^2} f_{a, \sigma^2}(\xi_1) = \frac{1}{\sqrt{2\pi\sigma}} \exp\left\{-\frac{(\xi_1 - a)^2}{2\sigma^2}\right\}$$

$$\int_{\mathbb{R}_n} T(x) \cdot \frac{\partial}{\partial \theta} f(x, \theta) dx = M\left(T(\xi) \cdot \frac{\partial}{\partial \theta} \ln L(\xi, \theta)\right) = \int_{\mathbb{R}_n} \frac{\partial}{\partial \theta} T(x) f(x, \theta) dx$$

$$\int_{\mathbb{R}_n} T(x) \cdot \left(\frac{\partial}{\partial \theta} \ln L(x, \theta)\right) \cdot f(x, \theta) dx = \int_{\mathbb{R}_n} T(x) \cdot \left(\frac{\frac{\partial}{\partial \theta} f(x, \theta)}{f(x, \theta)}\right) \cdot f(x, \theta) dx$$

$$\frac{\partial}{\partial \theta} M T(\xi) = \frac{\partial}{\partial \theta} \int_{\mathbb{R}_n} T(x) f(x, \theta) dx = \int_{\mathbb{R}_n} \frac{\partial}{\partial \theta} T(x) f(x, \theta) dx$$

$$\left\{ \frac{(\xi_1 - a)^2}{2\sigma^2} \right\} \frac{\partial}{\partial a} \ln f_{a, \sigma^2}(\xi_1) = \frac{(\xi_1 - a)}{\sigma^2}$$

Information Retrieval

Information Retrieval (IR)

There is a huge quantity of text, audio, video, and other documents available on the Internet, on about any subject. Users need to be able to find relevant information to satisfy their particular information needs. There are two ways of searching for information: to use a search engines or to browse directories organized by categories (such as Yahoo Directories). There is still a large part of the Internet that is not accessible (for example private databases and intranets).

Information retrieval (IR) is the task of representing, storing, organizing, and offering access to information items. IR is different from data retrieval, which is about finding precise data in databases with a given structure. In IR systems, the information is not structured, it is contained in free form in text (webpages or other documents) or in multimedia content.

Information retrieval vs. Data retrieval ?

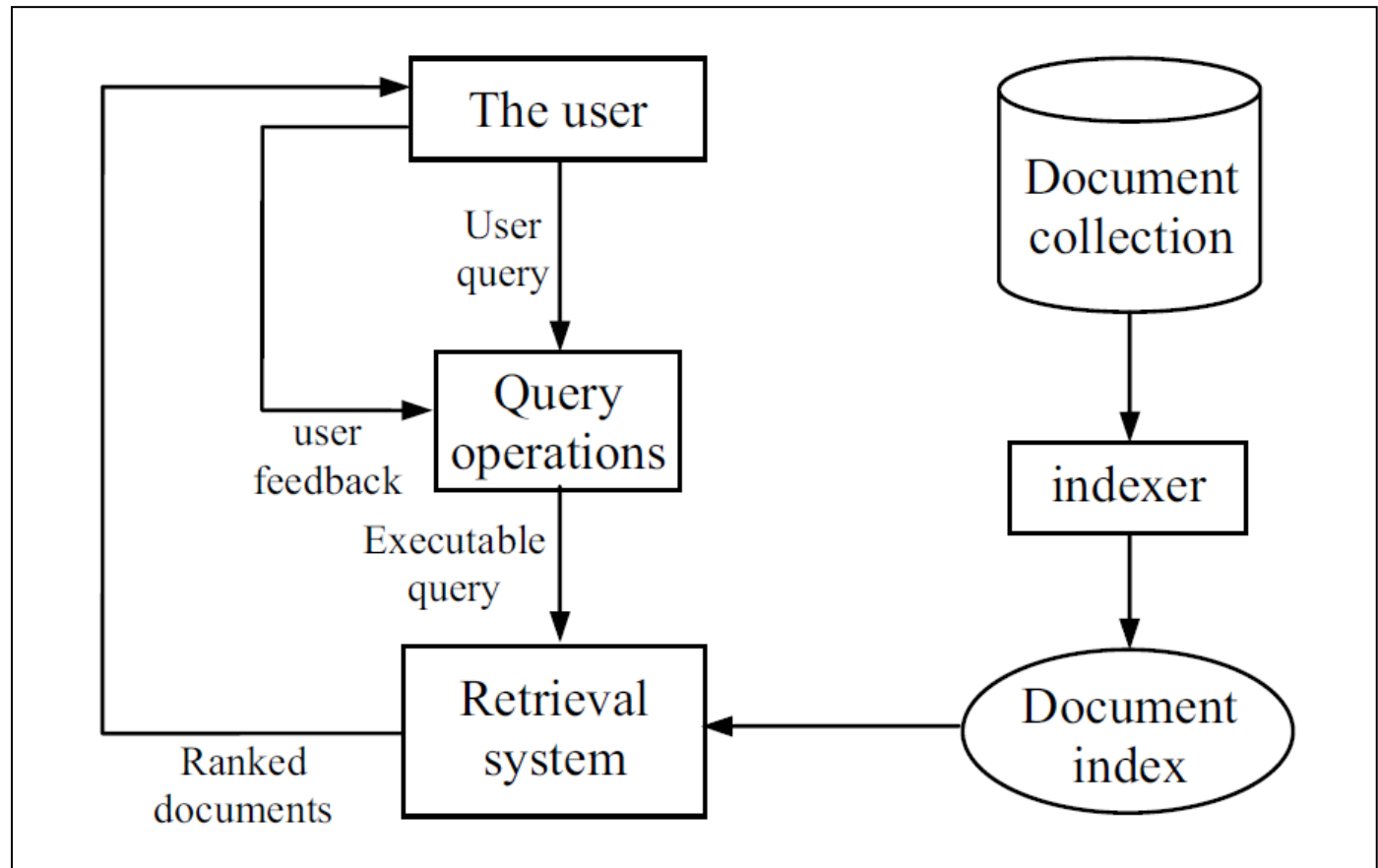


Source: *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data* by Bing Liu

Information Retrieval (IR)

Information retrieval (IR) is the study of helping users to find information that matches their information needs. Technically, IR studies the acquisition, organization, storage, retrieval, and distribution of information. Historically, IR is about document retrieval, emphasizing document as the basic unit.

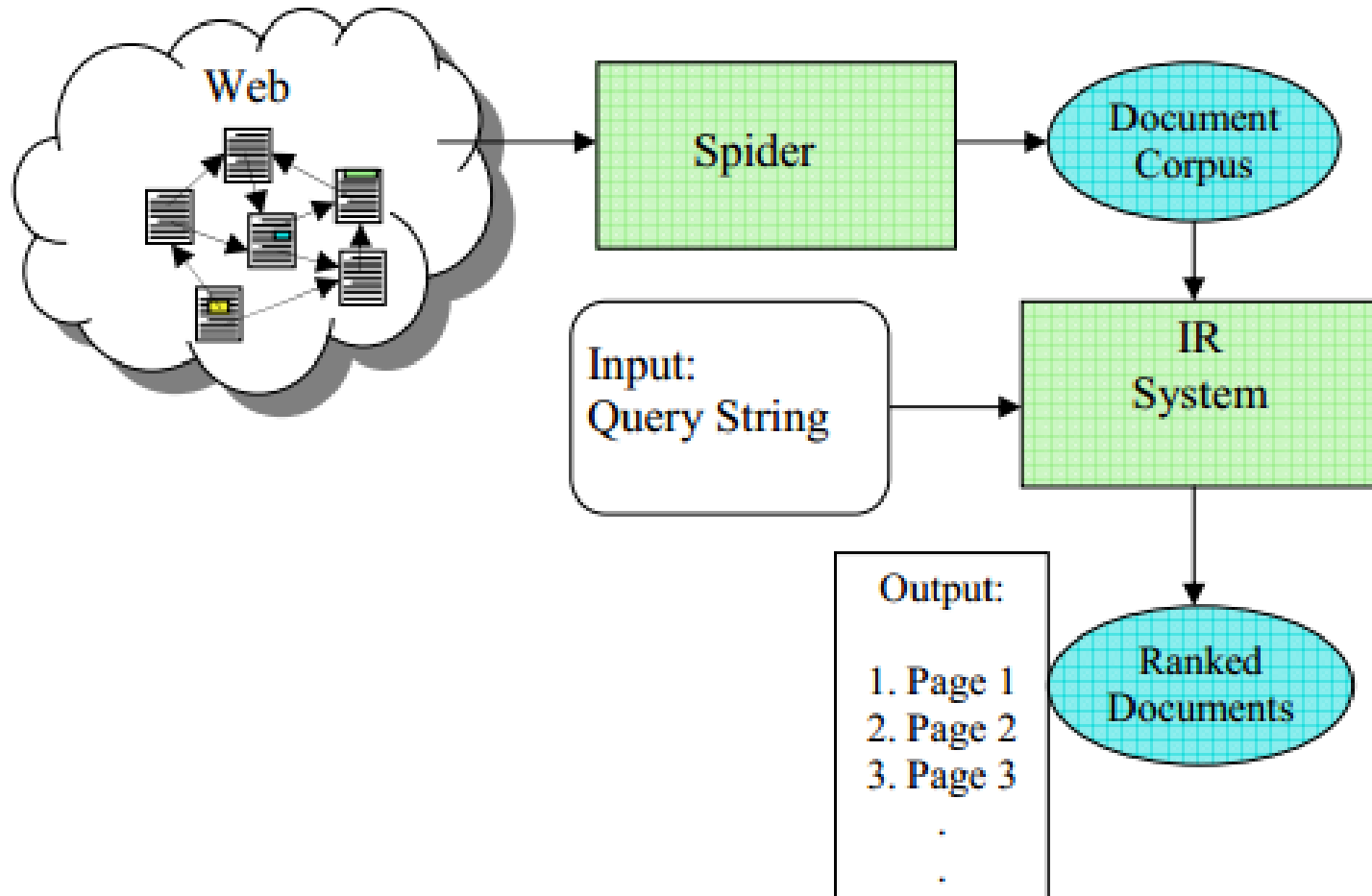
A general IR
system
architecture



Source: Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data by Bing Liu



The simplified architecture of a search engine



Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf





Web Crawler

The Web Crawler **collects documents** from the web to build a **searchable index** for the search engine.

WWW



The Web Crawler identifies all the **hyperlinks** in the pages and adds them to the list of URLs to visit



Search engine query



1. The web server sends the **query** to the index servers that tell which pages contain the words that match the query.
2. The query travels to the **doc servers**, which actually retrieve the stored documents. Snippets are generated to describe each search result.

The Web Crawler continuously populate the **index servers** (where the content is similar to the index in the books).



3. The **search results** are returned to the user.

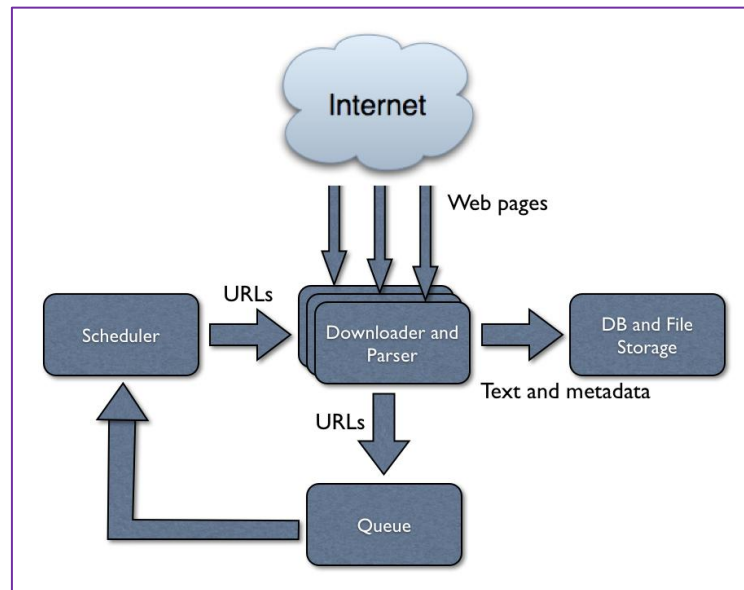
Web Crawling

Web crawlers, also known as spiders or robots, have the task to collect webpages to build the text collection for the IR system. The text is extracted from the **HTML code of the webpages**. Some information related to the HTML format may be stored too. **For example, text in headings or in bold font can be given higher weight than the rest of the text.**

A crawler starts with one or more http addresses (a set of root URLs), and follows all the links on these pages recursively, to find additional pages. It can proceed by **depth-first searching** (follow the first link in a page and all the links in the new pages that it leads to, then come back to follow the rest of the links in the current page) or by **breadth-first searching** (follow all the links in the page for one step, then the links in the pages they point to, for one step, etc.).

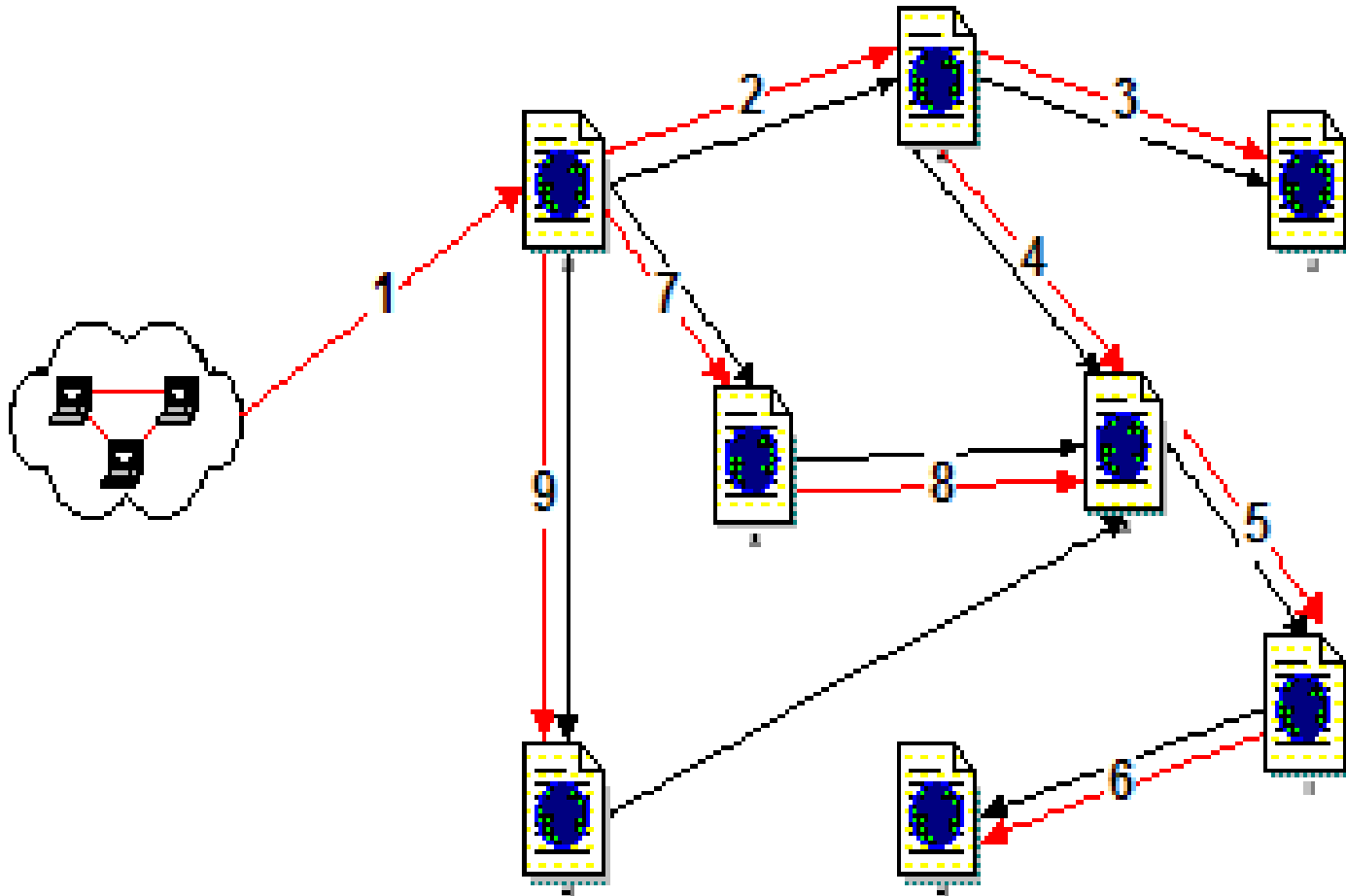
Source:

http://www.site.uotta.ca/~diana/csi4107/IR_draft.pdf



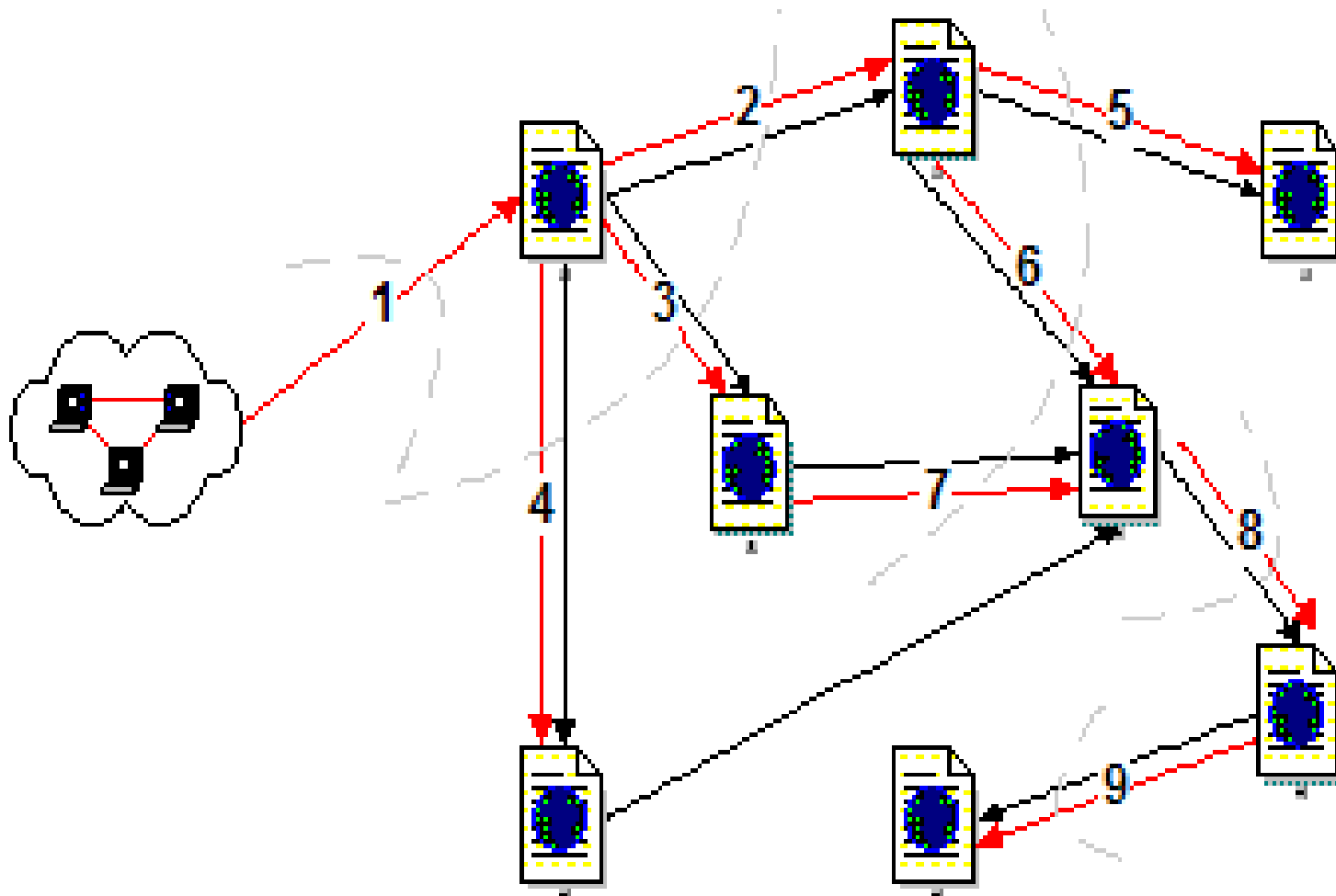
Depth First

Follow the first link in a page and all the links in the new pages that it leads to, then come back to follow the rest of the links in the current page



Breadth First

Breadth-first has the advantage that it explores uniformly outward from the root page but requires memory to store all the links waiting to be traversed on the previous level (exponential in the depth of the links structure). **It is the standard spidering method.**



Web Crawling

It is the standard spidering method.

Pseudocode

BFS:

```
1 procedure BFS(G,v):
2   create a queue Q
3   enqueue v onto Q
4   mark v
5   while Q is not empty:
6     t ← Q.dequeue()
7     for all edges e in G.adjacentEdges(t) do
8       u ← G.adjacentVertex(t,e)
9       if u is not marked:
10         mark u
11         enqueue u onto Q
```

BFS (Adpated for Web Crawling):

```
1 procedure BFS(array i of seed urls):
2   create a queue Q
3   enqueue seed urls v_i from array i onto Q
4   while Q is not empty:
5     t ← Q.dequeue()
6     Visit t and mark as visited
7     #Extract out outgoing links
8     for all outgoing links e in html of t do
9       if e is not visited:
10         enqueue e onto Q
```

Marking url as visited is done using a map/dictionary/hash.
It's used to ensure that we don't crawl the same page twice.

By changing queue Q into a priority queue PQ, one can do priority based crawling.
Crawl important pages first, the rest later.



Web Crawling

Depth-first requires less memory (linear in depth) but it might get “lost” pursuing a single thread. Both strategies can be easily implemented using a queue of links (URLs) to be visited next. How new links are added to the queue determines the search strategy. **FIFO (first in first out, append to end of the queue) gives breadth-first search. LIFO (last in first out, add to front of queue) gives depth-first search.** Heuristically ordering the queue gives a “focused crawler” that directs its search towards “interesting” pages. A spider needs to avoid visiting the same pages again when the links are circular; it needs to keep track of pages that were already visited.

DEPTH-FIRST VS. BREADTH-FIRST

- ◉ depth-first goes off into one branch until it reaches a leaf node
 - not good if the goal node is on another branch
 - neither complete nor optimal
 - uses much less space than breadth-first
 - much fewer visited nodes to keep track of
 - smaller fringe
- ◉ breadth-first is more careful by checking all alternatives
 - complete and optimal
 - very memory-intensive

Source: <http://www.sitepoint.com/articles/1111111/dfs-bfs/>



Web Crawling

To extract links from a webpage in order to collect candidate links to follow, HTML hyperlink fields are parsed. Here are two examples of hyperlinks:

```
<a href="https://cmpe.sjsu.edu/">  
<frame src="site-index.html">
```

If the URL is not specified, like in the last example, the link is relative to the current base URL. If a file name is not specified, a default name is used (such as index.html). The links are put into canonical form: the ending slash is removed, if there is one; internal references within the same page are removed, etc. Once the pages are collected, the text is extracted from the HTML documents, to be processed by the IR system.

Robot exclusion protocols are used to prevent certain sites or webpages from being indexed by Web crawlers. Web sites and pages can specify that robots should not crawl or index certain areas, by using the Robots Exclusion Protocol or the robots meta tag. The second one is newer and less well-adopted than the first one. These standards are conventions to be followed by “good robots”. They cannot be enforced, but companies have been prosecuted for “disobeying” these conventions and “trespassing” on private cyberspace.



Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf

What is HTML?

HTML is the language for describing the structure of Web pages. HTML gives authors the means to:

- Publish online documents with headings, text, tables, lists, photos, etc.
- Retrieve online information via hypertext links, at the click of a button.
- Design forms for conducting transactions with remote services, for use in searching for information, making reservations, ordering products, etc.
- Include spread-sheets, video clips, sound clips, and other applications directly in their documents.

With HTML, authors describe the structure of pages using markup. The elements of the language label pieces of content such as “paragraph,” “list,” “table,” and so on.

Source: <http://www.w3.org/standards/webdesign/htmlcss#whathtml>



History

The HyperText Markup Language (HTML) is the publishing language of the World Wide Web. The first version of HTML was described by Tim Berners-Lee in late 1991. For its first five years (1990-1995), HTML went through a number of revisions and experienced a number of extensions, primarily hosted first at CERN, and then at the IETF. With the creation of the W3C, HTML's development changed venue again. A first abortive attempt at extending HTML in 1995 known as HTML 3.0 then made way to a more pragmatic approach known as HTML 3.2, which was completed in 1997. HTML4 followed, reaching completion in 1998.

Version	Published year
HTML+	1993
HTML2.0	1995
HTML3.2	1997
HTML4.01	1999

Source: http://www.w3.org/wiki/HTML/Training/What_is_HTML%3F

Chandrasekar Vuppalapati, Professor, MS Software Engineering,
Computer Engineering, San Jose State University.



HTML File

```
1 <!doctype html>
<html lang="en"> 2
<head> 3
  <meta charset="utf-8">
  <title>Sample Web page</title> 4
  <script src="sample.js"> </script>
</head> 5
<body>
  <p>Hello!</p>
</body>
</html>
```

The DOCTYPE must be specified, and be top in the HTML document. That is, the DOCTYPE exists before the <html> start tag. The DOCTYPE declaration is <!DOCTYPE html> and is case-insensitive in the HTML syntax.

3

The head element represents a collection of metadata for the Document.

- The <head> ... </head> contains the title, and information on style sheets and scripts.
- Contents in the head tag are not displayed on a Web browser.

Source: http://www.w3.org/wiki/HTML/Training/Tag_syntax



Pre-defined Meta Data

Metadata Name	Description
application name	The name of the web application that the current page is part of
author	The name of the author of the current page
description	A description of the current page
generator	The name of the software that generated the HTML (this is usually used when using some kind of server framework to generate HTML pages, such as Ruby on Rails, ASP.NET, etc.)
keywords	A set of comma-separated strings that describe the content of the page



Source: http://www.w3.org/wiki/HTML/Training/Tag_syntax

Chandrasekar Vuppapapati, Professor, MS Software Engineering,
Computer Engineering, San Jose State University.

SEO

What is Search Engine Optimization and why should I care about it?

What is Search Engine Optimization?

Search Engine Optimization (SEO) is the process of improving internal and external aspects of a website, or Web page, to increase its organic visibility for search engines. SEO involves editing the website's HTML code and content to make it more search engine friendly, and then promoting the site to increase its relevance on the Web.

Once search engines are alerted to your website's presence, they scan the code and content of your site and index the information. Search engines analyze the website content to determine when and where your website displays on a search-engine result page.

The page content (text displaying on a Web page) should be inviting, comprehensive, and — within reasonable limits — contain as many of the site's keywords as possible.

Some search engines, including Google®, pay particular attention to the number of websites linking to your website when determining the importance and ranking of your site. These external links are called back links.



Source: <https://support.godaddy.com/help/article/6622/what-is-search-engine-optimization-and-why-should-i-care-about-it>

What is Search Engine Optimization and why should I care about it?

Why should I care about SEO?

You've heard the phrase "Location, location, location!" and its importance when shopping for business real estate. The same applies to the placement of your website on search engine result pages. According to research, the higher your site is listed as a search result, the more traffic you're going to get.

Quality traffic means increased revenue and more publicity. Search engine traffic can lead to an organization's success or failure.

The goal of SEO is to land your website in the top few pages of search engine's results page. This is not easy. It takes a lot of time and constant tweaking to increase your search engine rankings.

To assist you with the process, Search Engine Visibility guides you through optimizing your website for search engine inclusion. Search Engine Visibility analyzes your site by applying various rules based on what search engines see when they visit your site. Search Engine Visibility reports the results of the analysis, and suggests measures you can take to improve the optimization of your site.



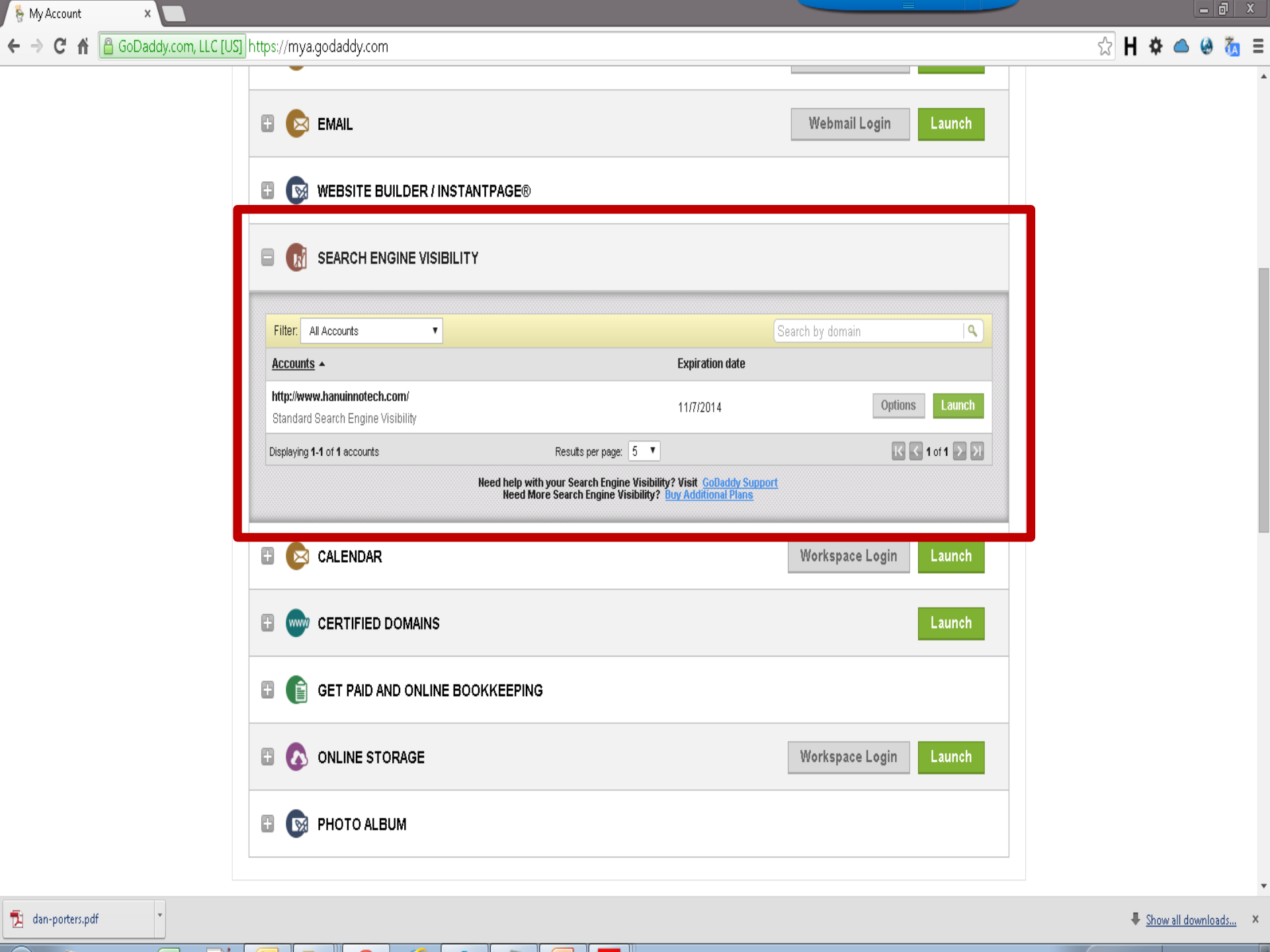
Source: <https://support.godaddy.com/help/article/6622/what-is-search-engine-optimization-and-why-should-i-care-about-it>

Search Engine Visibility FAQ

- **How does Search Engine Visibility help me optimize my site?**
- **Will my website be ranked high in search engines if I use Search Engine Visibility?**
- **Can I Pay to Guarantee a Search Engine Listing?**
- **What are keywords and why do I need them?**
- **How often should I submit my sitemap?**
- **What's the difference between robots.txt and Meta Tags?**
- **Does Search Engine Visibility optimize my site for me?**



Source: <https://support.godaddy.com/help/article/6622/what-is-search-engine-optimization-and-why-should-i-care-about-it>



EMAIL

Webmail Login

Launch



WEBSITE BUILDER / INSTANTPAGE®



SEARCH ENGINE VISIBILITY

Filter: All Accounts

Search by domain



Accounts ▲

Expiration date

http://www.hanuinnotech.com/
Standard Search Engine Visibility

11/7/2014

Options

Launch

Displaying 1-1 of 1 accounts

Results per page: 5

1 of 1

Need help with your Search Engine Visibility? Visit [GoDaddy Support](#)
Need More Search Engine Visibility? [Buy Additional Plans](#)



CALENDAR

Workspace Login

Launch



CERTIFIED DOMAINS

Launch



GET PAID AND ONLINE BOOKKEEPING



ONLINE STORAGE

Workspace Login

Launch



PHOTO ALBUM

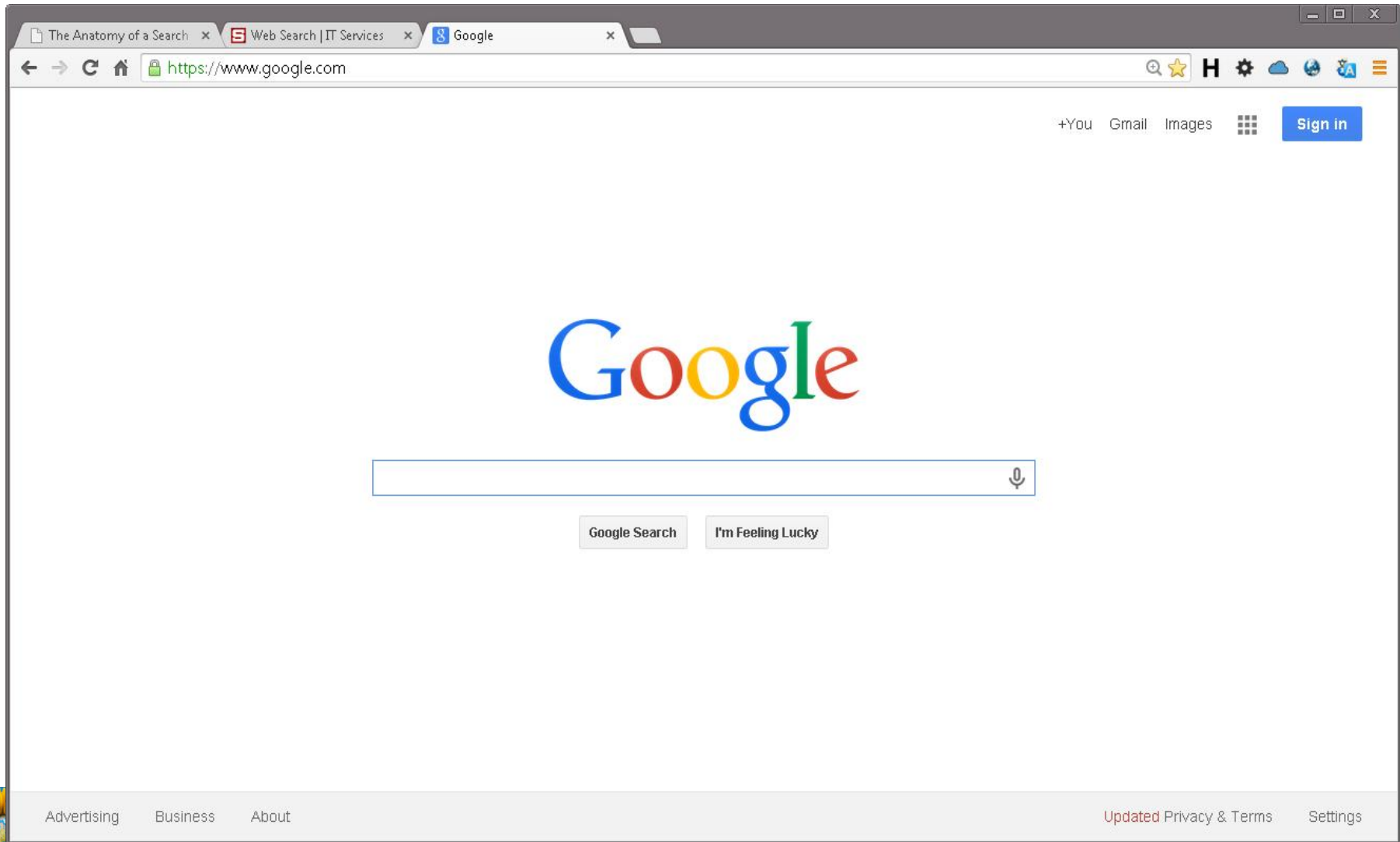
Keeping Spidered Pages Up-to-Date

The Web is very dynamic: there are many new pages, updated pages, deleted pages, etc. A search engine needs to periodically check spidered pages for updates and deletions. A spider could look in the HTML head information (e.g. meta tags on the last update) to determine if the page has changed, and only reload the entire the page if needed. It could track how often each page is updated and preferentially return to pages which are historically more dynamic. It could preferentially update pages that are accessed more often to optimize the freshness of popular pages.

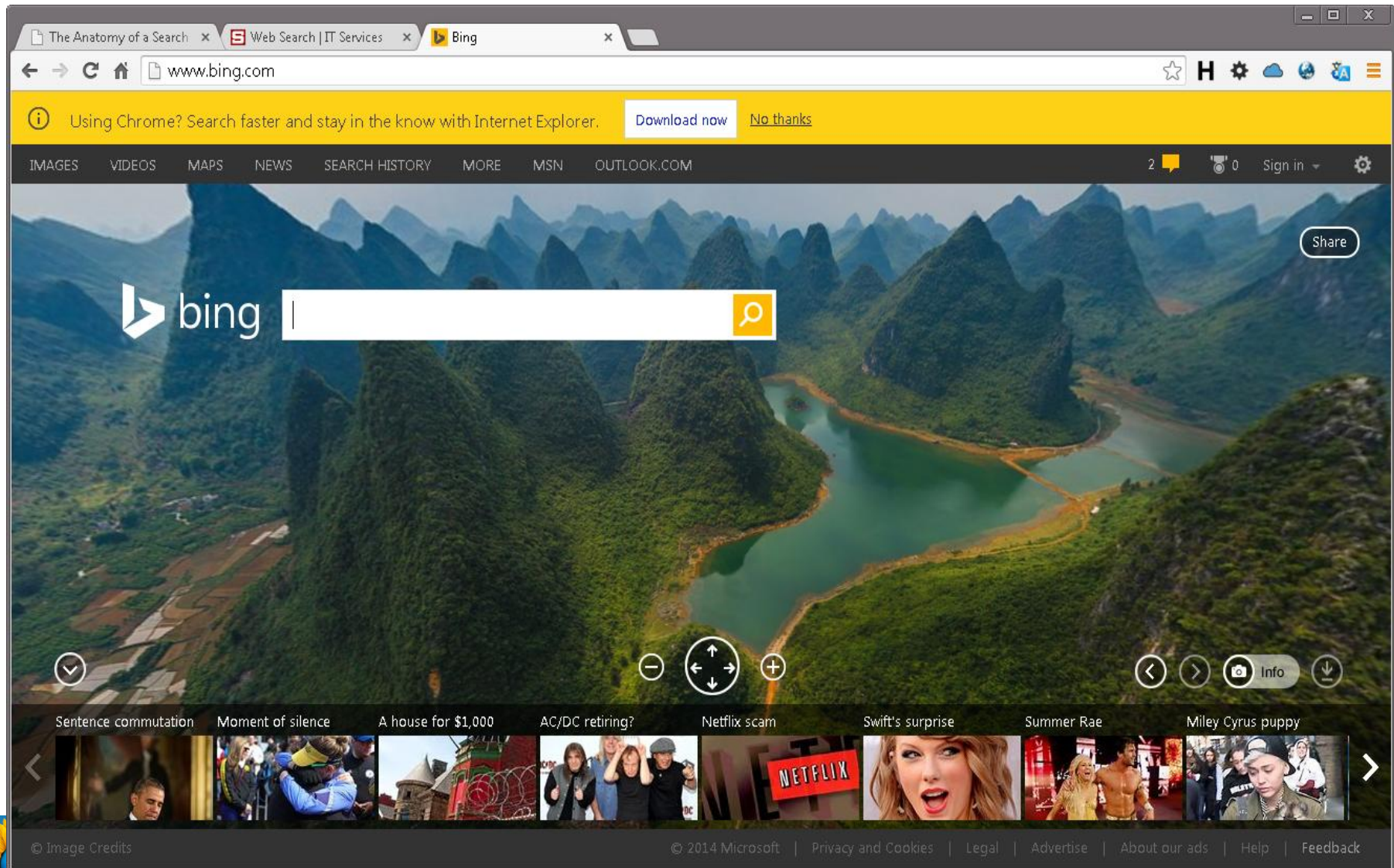


Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf

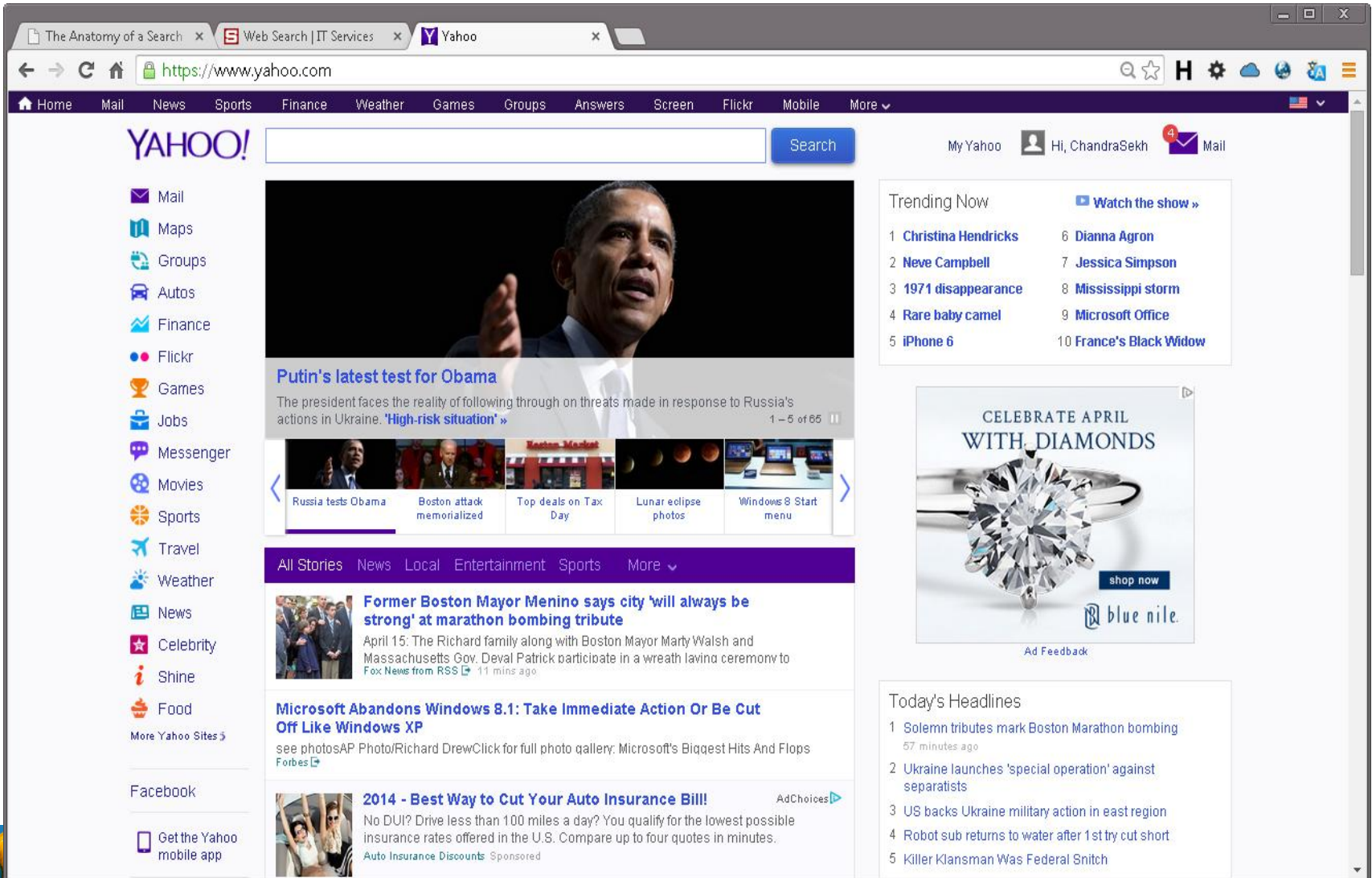
Search Engines



Search Engines



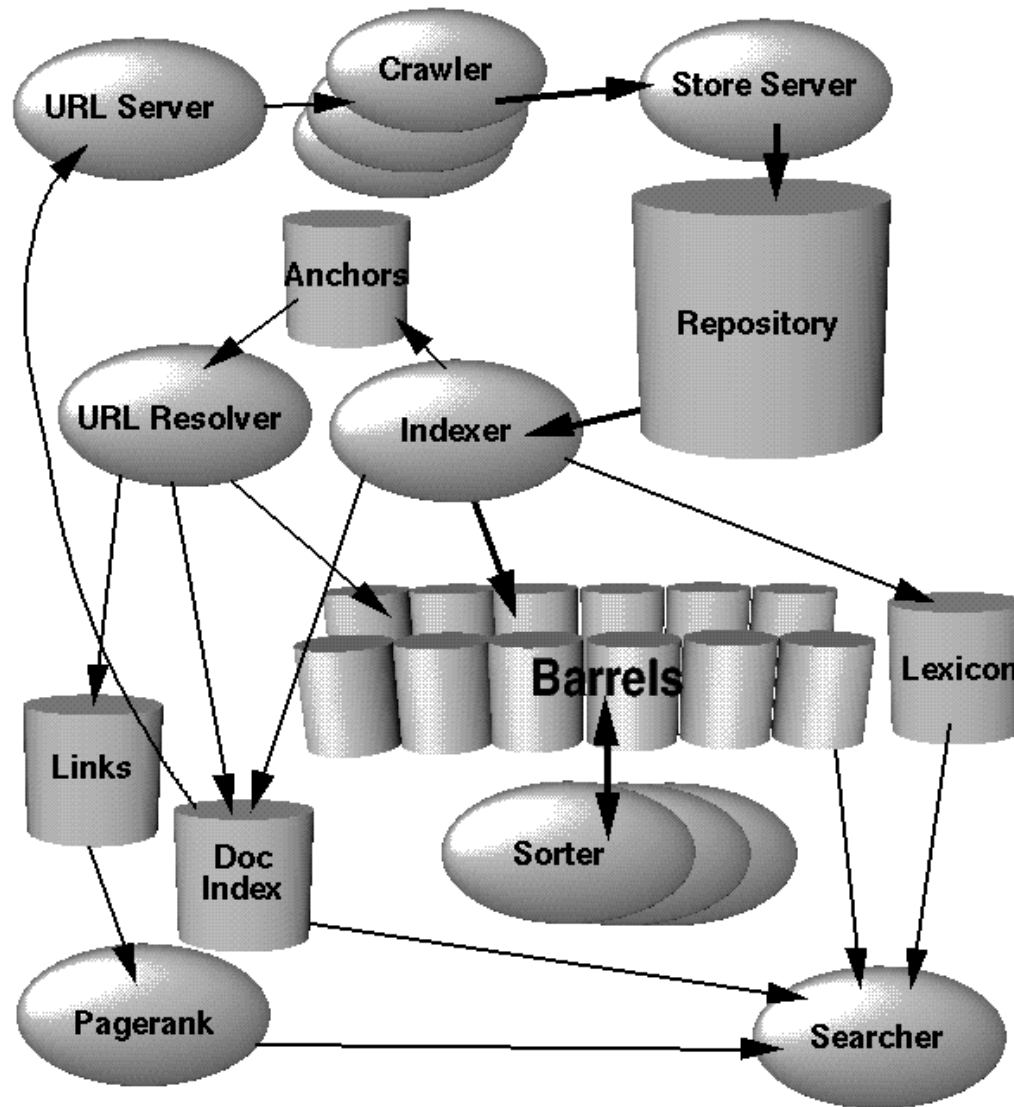
Search Engines



The screenshot shows the Yahoo! homepage with the following elements:

- Browser Tabs:** "The Anatomy of a Search", "Web Search | IT Services", and "Yahoo".
- Address Bar:** "https://www.yahoo.com".
- Navigation Menu:** Home, Mail, News, Sports, Finance, Weather, Games, Groups, Answers, Screen, Flickr, Mobile, More.
- Search Bar:** "YAHOO!" logo, a search input field, and a "Search" button.
- User Account:** "My Yahoo", "Hi, ChandraSekh", and a "Mail" icon with a notification badge.
- Left Sidebar:** Links to Mail, Maps, Groups, Autos, Finance, Flickr, Games, Jobs, Messenger, Movies, Sports, Travel, Weather, News, Celebrity, Shine, Food, and "More Yahoo Sites".
- Main Content Area:**
 - Featured Story:** "Putin's latest test for Obama" with a large image of Barack Obama. Subtext: "The president faces the reality of following through on threats made in response to Russia's actions in Ukraine. 'High-risk situation'".
 - News Carousel:** "Russia tests Obama", "Boston attack memorialized", "Top deals on Tax Day", "Lunar eclipse photos", "Windows 8 Start menu".
 - News Section:**
 - Former Boston Mayor Menino says city 'will always be strong' at marathon bombing tribute** (April 15: The Richard family along with Boston Mayor Marty Walsh and Massachusetts Gov. Deval Patrick participate in a wreath laying ceremony to Fox News from RSS 11 mins ago).
 - Microsoft Abandons Windows 8.1: Take Immediate Action Or Be Cut Off Like Windows XP** (see photosAP Photo/Richard DrewClick for full photo gallery: Microsoft's Biggest Hits And Flops Forbes).
 - 2014 - Best Way to Cut Your Auto Insurance Bill!** (No DUI? Drive less than 100 miles a day? You qualify for the lowest possible insurance rates offered in the U.S. Compare up to four quotes in minutes. Auto Insurance Discounts Sponsored).
- Right Sidebar:**
 - Trending Now:** A list of 10 trending topics: Christina Hendricks, Dianna Agron, Neve Campbell, Jessica Simpson, 1971 disappearance, Mississippi storm, Rare baby camel, Microsoft Office, iPhone 6, France's Black Widow.
 - Advertisement:** "CELEBRATE APRIL WITH DIAMONDS" featuring a diamond ring and the "blue nile" logo.
 - Today's Headlines:** A list of 5 headlines: Solemn tributes mark Boston Marathon bombing, Ukraine launches 'special operation' against separatists, US backs Ukraine military action in east region, Robot sub returns to water after 1st try cut short, Killer Klansman Was Federal Snitch.

Google Architecture Overview

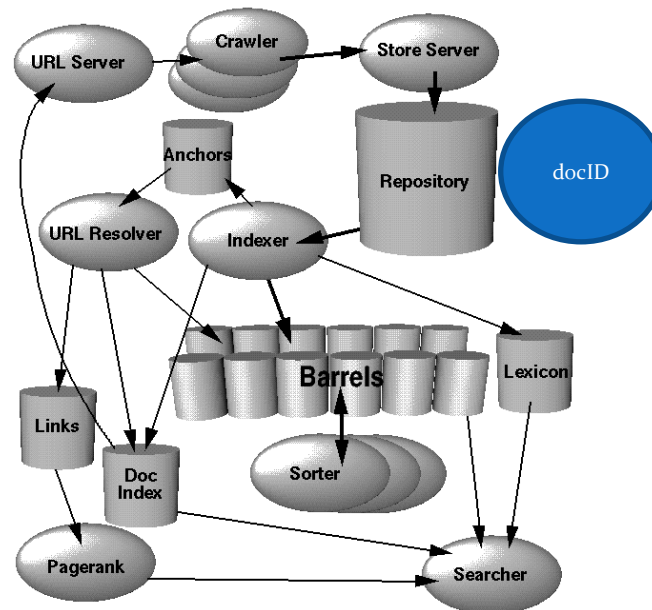


Source:
<http://infolab.stanford.edu/~backrub/google.html> by Sergey Brin and Lawrence Page



Most of Google is implemented in C or C++ for efficiency and can run in either Solaris or Linux.

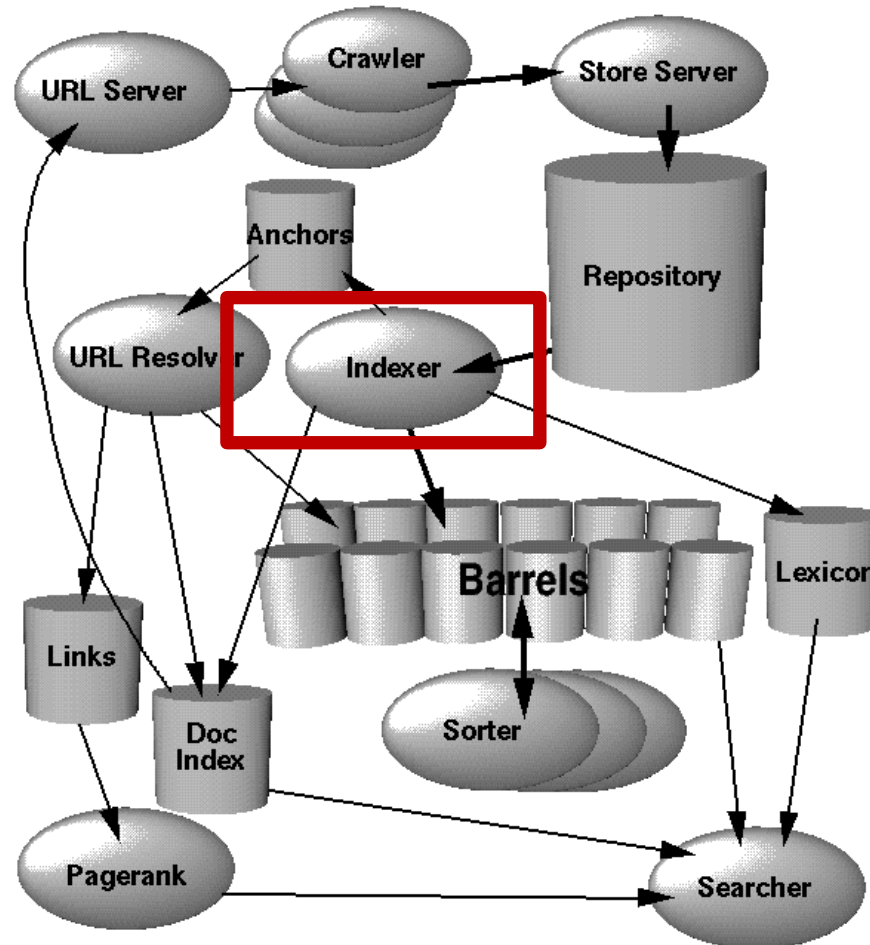
- In Google, the web crawling (downloading of web pages) is done by **several distributed crawlers**.
- There is a URLserver that sends lists of URLs to be fetched to the crawlers. The web pages that are fetched are then sent to the storeserver.
- The storeserver then compresses and stores the web pages into a repository. Every web page has an associated ID number called a **docID** which is assigned whenever a new URL is parsed out of a web page. The indexing function is performed by the indexer and the sorter.



Source: <http://infolab.stanford.edu/~backrub/google.html> by Sergey Brin and Lawrence Page

Google Architecture Overview

- The indexer performs a number of functions. It reads the repository, uncompresses the documents, and parses them. Each document is **converted into a set of word occurrences called hits**. The hits record the **word, position in document, an approximation of font size, and capitalization**.

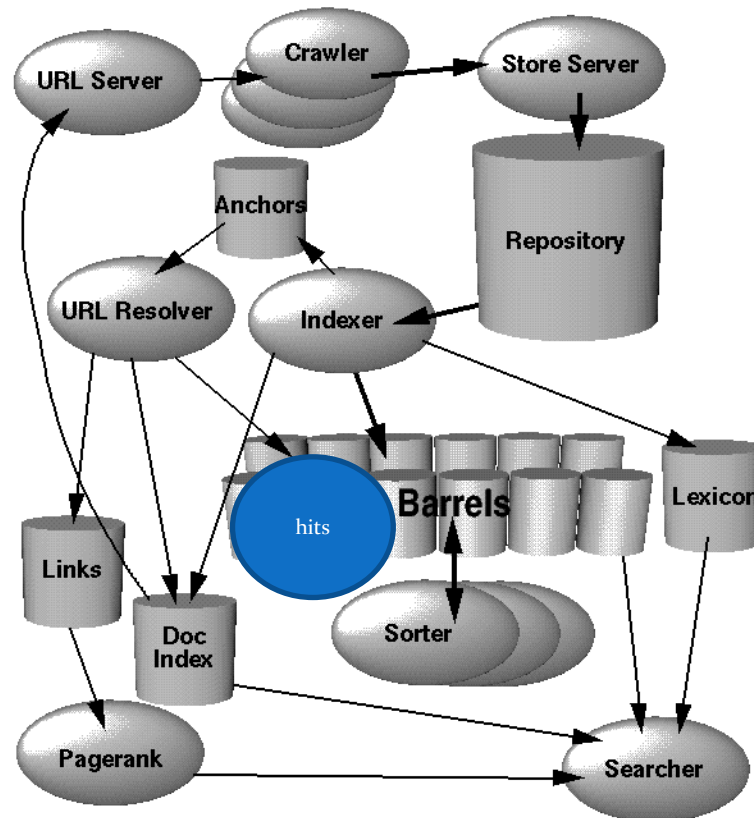


Source: <http://infolab.stanford.edu/~backrub/google.html> by Sergey Brin and Lawrence Page



Google Architecture Overview

- The indexer distributes these hits into a set of "barrels", creating a partially sorted forward index. The indexer performs another important function. It parses out all the links in every web page and stores important information about them in an anchors file. This file contains enough information to determine where each link points from and to, and the text of the link.

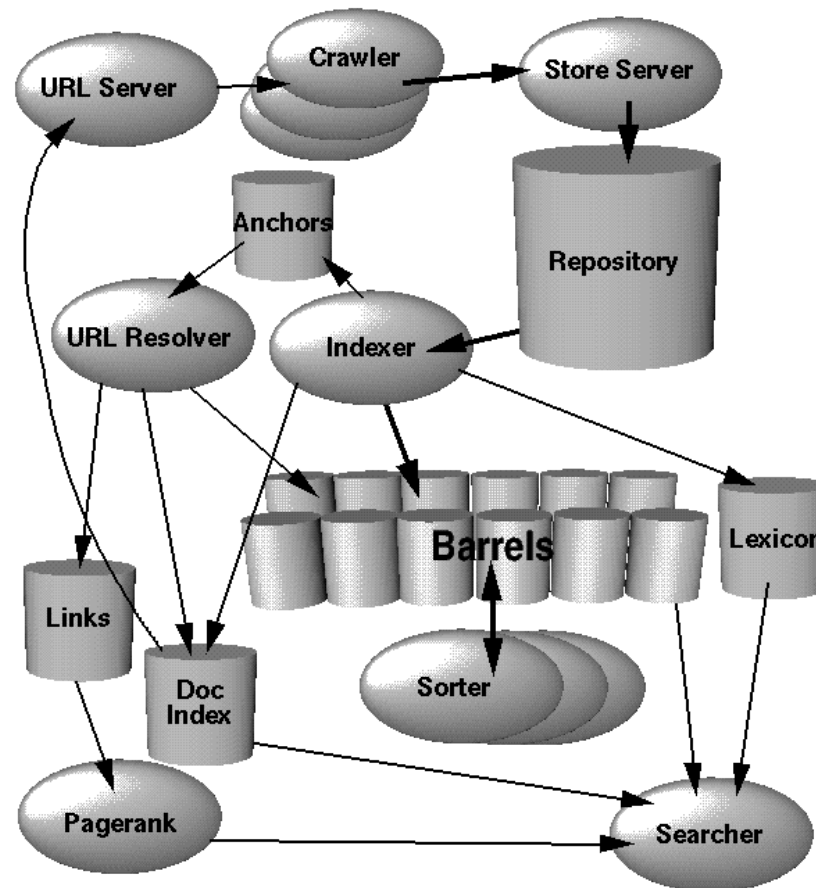


Source: <http://infolab.stanford.edu/~backrub/google.htm> by Sergey Brin and Lawrence Page



Google Architecture Overview

- The URLresolver reads the anchors file and converts relative URLs into absolute URLs and in turn into docIDs. It puts the anchor text into the forward index, associated with the docID that the anchor points to. It also generates a database of links which are pairs of docIDs. The links database is used to compute PageRanks for all the documents.



Source: <http://infolab.stanford.edu/~backrub/google.html> by Sergey Brin and Lawrence Page



Google Architecture Repository Data Structure

Repository: 53.5 GB = 147.8 GB uncompressed

sync	length	compressed packet
sync	length	compressed packet

...

Packet (stored compressed in repository)

docid	ecode	urlen	pagelen	url	page
-------	-------	-------	---------	-----	------

Source: <http://infolab.stanford.edu/~backrub/google.html> by Sergey Brin and Lawrence Page



Forward and Reverse Indexes and the Lexicon

Hit: 2 bytes

plain:	cap:1	imp:3	position: 12	
fancy:	cap:1	imp = 7	type: 4	position: 8
anchor:	cap:1	imp = 7	type: 4	hash:4 pos: 4

Forward Barrels: total 43 GB

docid	wordid: 24	nhits: 8	hit hit hit hit
	wordid: 24	nhits: 8	hit hit hit hit
	null wordid		
docid	wordid: 24	nhits: 8	hit hit hit hit
	wordid: 24	nhits: 8	hit hit hit hit
	wordid: 24	nhits: 8	hit hit hit hit
	null wordid		

...

Lexicon: 293MB

Inverted Barrels: 41 GB

wordid	ndocs	→	docid: 27	nhits:5	hit hit hit hit
wordid	ndocs	→	docid: 27	nhits:5	hit hit hit
wordid	ndocs	→	docid: 27	nhits:5	hit hit hit hit
		→	docid: 27	nhits:5	hit hit

...

Repository: 53.5 GB = 147.8 GB uncompressed

sync	length	compressed packet
sync	length	compressed packet

...

Packet (stored compressed in repository)

docid	ecode	urlen	pagelen	url	page
-------	-------	-------	---------	-----	------

Source: <http://infolab.stanford.edu/~backrub/google.html> by Sergey Brin and Lawrence Page



Case Study



Common Crawl

[Home](#)[About »](#)[Get Started](#)[Blog](#)[Jobs](#)[Donate](#)[Media](#)

Common Crawl is a non-profit foundation dedicated to providing an open repository of web crawl data that can be accessed and analyzed by everyone.

Fresh Data Available!

The latest dataset is from March 2014, contains approximately 2.8 billion webpages and is located in [Amazon Public Data Sets](#) at `/common-crawl/crawl-data/CC-MAIN-2014-10`.

[Our Work](#)[Team](#)[Get Started](#)[GitHub](#)

Common Crawl

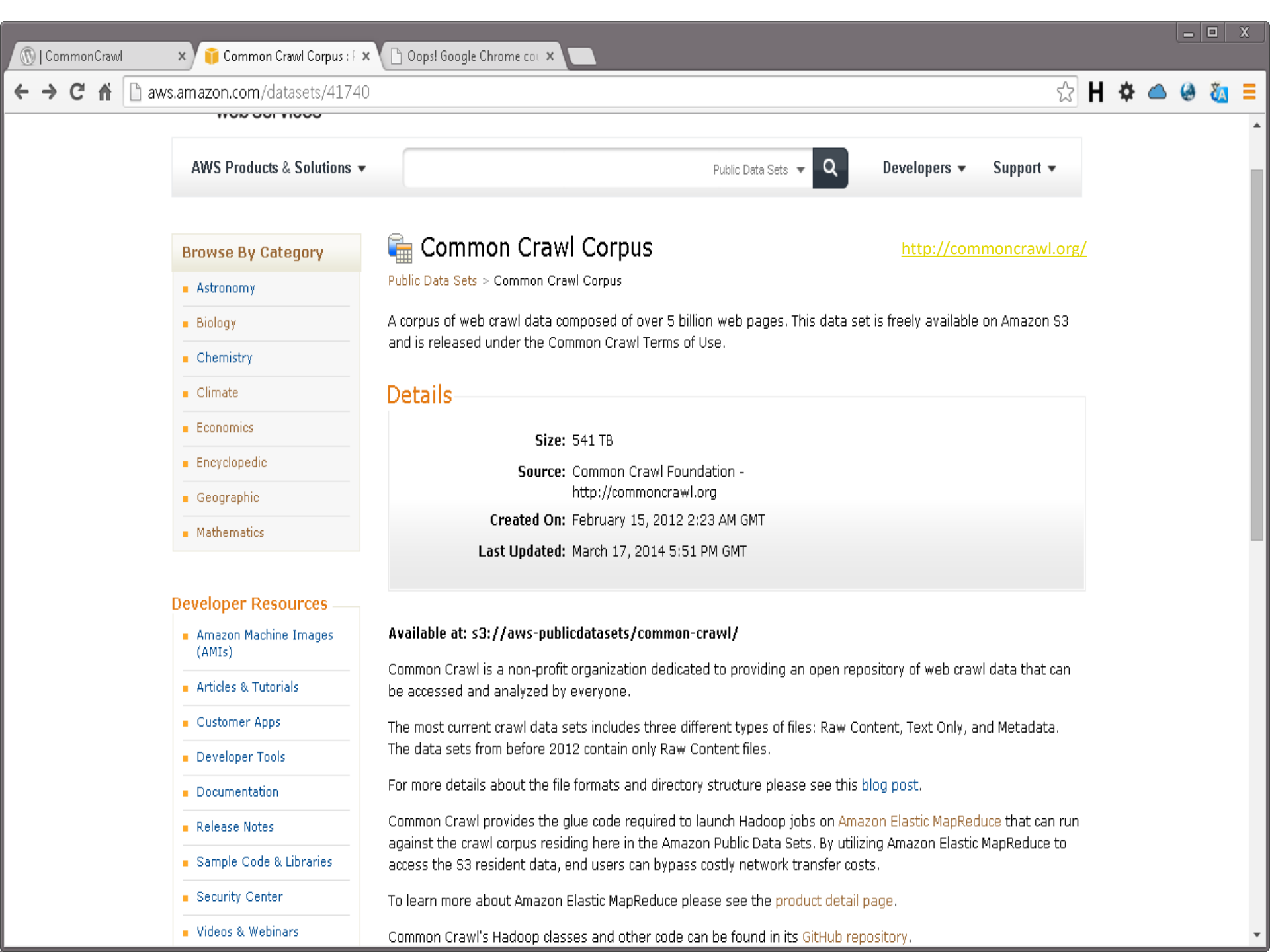
Common Crawl Foundation is a California 501(c)(3) registered non-profit founded by Gil Elbaz with the goal of democratizing access to web information by producing and maintaining an open repository of web crawl data that is universally accessible and analyzable.

As the largest and most diverse collection of information in human history, the web grants us tremendous insight if we can only understand it better. For example, web crawl data can be used to spot trends and identify patterns in politics, economics, health, popular culture and many other aspects of life. It provides an immensely rich corpus for scientific research, technological advancement, and innovative new businesses. It is crucial for our information-based society that the web be openly accessible to anyone who desires to utilize it.

We strive to be transparent in all of our operations and we support nofollow and robots.txt. For more information about the ccBot, please see [FAQ](#). For more information on Common Crawl data and how to access it, please see [Data](#). For access to our open source code, please see our [GitHub repository](#).



Source: <http://commoncrawl.org/>



AWS Products & Solutions ▾

Public Data Sets ▾



Developers ▾

Support ▾

Browse By Category

- [Astronomy](#)
- [Biology](#)
- [Chemistry](#)
- [Climate](#)
- [Economics](#)
- [Encyclopedic](#)
- [Geographic](#)
- [Mathematics](#)

Developer Resources

- [Amazon Machine Images \(AMIs\)](#)
- [Articles & Tutorials](#)
- [Customer Apps](#)
- [Developer Tools](#)
- [Documentation](#)
- [Release Notes](#)
- [Sample Code & Libraries](#)
- [Security Center](#)
- [Videos & Webinars](#)

Common Crawl Corpus

<http://commoncrawl.org/>

Public Data Sets > Common Crawl Corpus

A corpus of web crawl data composed of over 5 billion web pages. This data set is freely available on Amazon S3 and is released under the Common Crawl Terms of Use.

Details

Size: 541 TB

Source: Common Crawl Foundation -
<http://commoncrawl.org>

Created On: February 15, 2012 2:23 AM GMT

Last Updated: March 17, 2014 5:51 PM GMT

Available at: `s3://aws-publicdatasets/common-crawl/`

Common Crawl is a non-profit organization dedicated to providing an open repository of web crawl data that can be accessed and analyzed by everyone.

The most current crawl data sets includes three different types of files: Raw Content, Text Only, and Metadata. The data sets from before 2012 contain only Raw Content files.

For more details about the file formats and directory structure please see this [blog post](#).

Common Crawl provides the glue code required to launch Hadoop jobs on [Amazon Elastic MapReduce](#) that can run against the crawl corpus residing here in the Amazon Public Data Sets. By utilizing Amazon Elastic MapReduce to access the S3 resident data, end users can bypass costly network transfer costs.

To learn more about Amazon Elastic MapReduce please see the [product detail page](#).

Common Crawl's Hadoop classes and other code can be found in its [GitHub repository](#).



Semantics3

[Use Cases](#)[Quick Start](#)[Pricing](#)[FAQ](#)[Blog](#)[Sign Up](#)[Login](#)

Announcing our partnership with Two Tap!

[Blog Post](#)[TwoTap Website](#)

Curated Database of Products and Prices

2 billion prices from 35 million products across 22,000 retailers, refreshed constantly

[Get Instant Access](#)[\[Run API Queries \]](#)[About the Data](#)[Product Discovery](#)[Price Comparisons](#)

zopimchat



Chat with us

Type your message here

[Send](#)

Semantic S3

Semantics3 aims to organize the world's e-commerce data. Having gathered billions of prices, tens of millions of products and thousands of merchants, we're well underway. Our goal is to curate every price and every product on the entire web in our database and become the default destination for all product and pricing information.

Semantics3 was founded in 2012 with the aim of making structured data from across the web easily accessible to all. **We specialize in crawling, curating and merging data at scale from a variety of sources.** This data is indexed along several dimensions and made available for developers to harness and innovate upon by means of an API. Our primary offering is our Products Data API. With this API, developers can track prices across a variety of products, easily build shopping comparison engines, enrich e-commerce stores with rich metadata and more. We subscribe to the philosophy that more data beats fancy algorithms and aim to help businesses tap into the large amounts of structured data available on the web.



Source: <http://infolab.stanford.edu/~backrub/google.html>

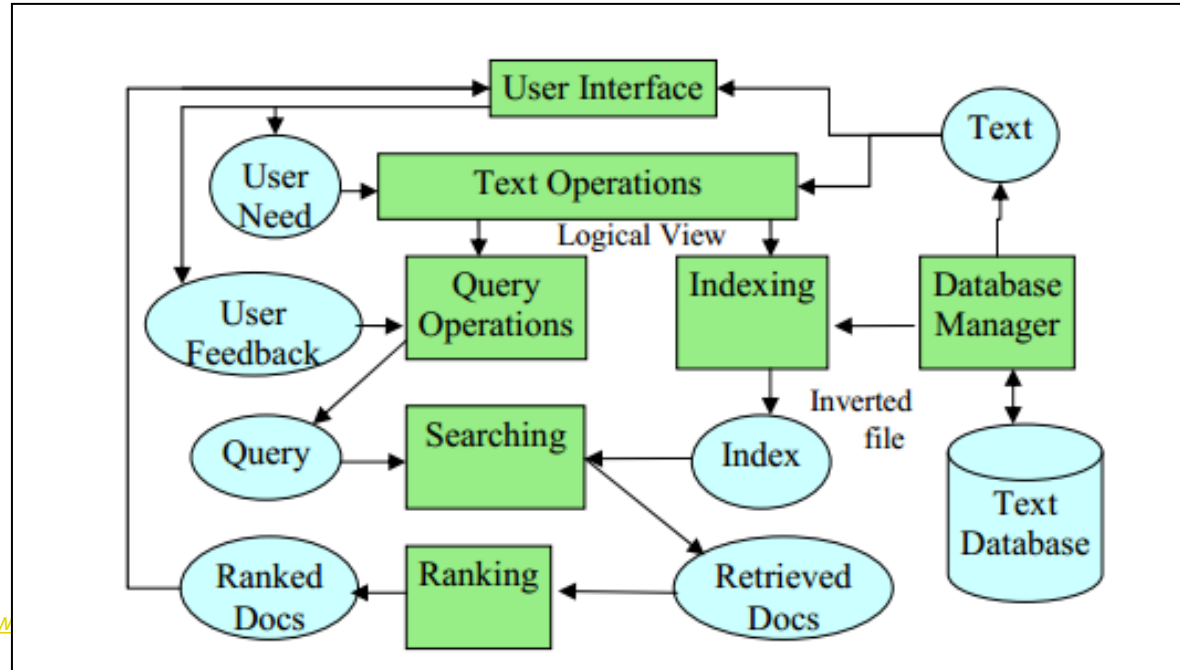
Information Retrieval

Information Retrieval (IR)

Following figure presents a more detailed view of the architecture of an IR system.

Text Operations are used to preprocess the documents collections and to **extract index words**.

- The indexing module constructs an inverted index from words to document pointers.
- The searching module retrieves documents that contain given query words, using the inverted index.
- The ranking module scores all the retrieved documents according to a relevance metric.
- The user interface manages interaction with the user: the input of the query and the output of the ranked documents, including the visualization of results.
- The query operations can transform the query in order to improve retrieval (query expansion using synonyms from a thesaurus, query transformation using relevance feedback).



Source: <http://www>

Preprocessing the document collection

There are several preprocessing steps needed to prepare the document collection for the IR task.

- The first step is to filter out unwanted characters and markup (e.g. HTML tags, punctuation, numbers, etc.).
- Then the text needs to be broken into tokens (keywords) by using as delimiters white space and punctuation characters. This is not quite as straightforward as it seems, since words in texts are not always clearly delimited (for example, if the text is You can't do this, you can consider the apostrophe as a word separator to get two words can and t, or ignore it as separator and consider one word can't, or expand the contracted form into two words can and not and use the white space as separator).
- The keywords can be used as they are, or they can be transformed into a base form, for example nouns in the singular form, verbs in the infinitive form, etc. (e.g., books becomes book, talked becomes talk).
- A common approach is to stem the tokens to “stem” forms. For example, computational becomes comput and computing becomes comput. **Stemming** the terms before building the inverted index has the advantage that it reduces the size of the index, and allows for retrieval of webpages with various inflected forms of a word (for example, when searching for webpages with the word computation, the results will include webpages with computations and computing).
- Stemming is easier to do than computing base forms, because stemmers remove suffixes, without needing a full dictionary of words in a language. **A popular and fast stemmer is Porter's stemmer**



Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf

Preprocessing the document collection

- Another useful preprocessing step is to remove very frequent words that appear in most of the documents and do not bear any meaningful content. They are called stopwords (e.g., a, the, it, of, could, etc.). An example of stopwords list can be found at:

<http://www.lextek.com/manuals/onix/stopwords1.html>

Important phrases composed of two or more words could also be detected to be used as keywords (possibly using a domain specific dictionary, or using a statistical method for analyzing the text collection in order to detect sequences of words that appear together very often).

Now the text is ready for the next step, building the inverted index that stores for each keyword a list of documents that contain it, in order to allow for fast access during the retrieval step.



Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf

The Porter Stemmer

Daniel Waegel
CISC889 / Fall 2011

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>

Stemming approaches

- Many different approaches:
 - Brute force look up
 - Suffix, affix stripping
 - Part-of-speech recognition
 - Statistical algorithms (n-grams, HMM)
- Porter stemmer utilizes suffix stripping
- It does not address prefixes

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>

Porter Stemmer Steps

- Step 1: Gets rid of plurals and -ed or -ing suffixes
- Step 2: Turns terminal y to i when there is another vowel in the stem
- Step 3: Maps double suffixes to single ones: -ization, -ational, etc.
- Step 4: Deals with suffixes, -full, -ness etc.
- Step 5: Takes off -ant, -ence, etc.
- Step 6: Removes a final -e

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>

Step 1

Gets rid of plurals and -ed or -ing suffixes

```
if b[k] == 's'
    if ends("sses")
        k -= 2
    else if ends("ies")
        setto("i")
    else if b[k-1] != 's'
        k--
if ends("eed")
    if m() > 0
        k--
else if ends("ed") || ends("ing") &&
vowelinstem()
    k = j
    if ends("at")
        setto("ate")
    else if ends("bl")
        setto("ble")
    else if ends("iz")
        setto("ize")
    else if doublec(k)
        k--
        int ch = b[k]
        if ch=='l' || ch=='s' || ch=='z'
            k++
else if (m() == 1 && cvc(k))
    setto("e")
```

- Possesses → possess
- Ponies → poni
- Operatives → operative
- **Markedly → markedly**
- Interesting → interest
- **Confess → confess**
- Consumables → consumable
- Realizes → realize
- Infuriating → Infuriate
- Fables → fable
- Fated → fate

Step 2

Turns terminal y to i when there is another vowel in the stem

```
if ends("y") && vowelinstem()  
    b[k] = 'i'
```

- Coolly → coolli

- Furry → furri

- Fry → fry

- Grey → grei

- **Interestingly →
Interestinglyli**

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>

Step 3

Maps double suffixes to single ones

```
switch b[k-1]
case 'a':
    if ends("ational") -> r("ate")
    if ends("tional") -> r("tion")
case 'c':
    if ends("enci")) -> r("ence")
    if ends("anci")) -> r("ance")
case 'e':
    if ends("izer") -> r("ize")
case 'l':
    if ends("bli") -> r("ble")
    if ends("alli") -> r("al")
    if ends("entli") -> r("ent")
    if ends("eli") -> r("e")
    if ends("ousli") -> r("ous")
case 'o':
    if ends("ization") -> r("ize")
    if ends("ation") -> r("ate")
    if ends("ator") -> r("ate")
case 's':
    if ends("alism") -> r("al")
    if ends("iveness") -> r("ive")
    if ends("fulness") -> r("ful")
    if ends("ousness") -> r("ous")
case 't':
    if ends("aliti") -> r("al")
    if ends("iviti") -> r("ive")
    if ends("bliti") -> r("ble")
case 'g':
    if ends("logi") -> r("log")
```

- **Rational → rational**
- **Optional → option**
- **Operational → operate**
- **Possibly → possibli → possible**
- **Really → realli → realli**
- **Realization → realize**
- **Feudalism → feudal**
- **Playfulness → playful**
- **Liveness → liveness**

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>

Step 4

Deals with suffixes, -full, -ness etc.

```
switch b[k]
  case 'e':
    if ends("icate") -> r("ic")
    if ends("ative") -> r("")
    if ends("alize") -> r("al")
  case 'i':
    if ends("iciti") -> r("ic")
  case 'l':
    if ends("ical") -> r("ic")
    if ends("ful") -> r("")
  case 's':
    if ends("ness") -> r("")
```

- Authenticate → authentic
- Predicate → predic
- **Realize → realize**
- Felicity → felicitati → felic
- Practical → practic
- Playful → play
- **Gleeful → gleeful**
- Largeness → large

Step 5

Takes off -ant, -ence, etc.

```
switch b[k-1]
  case 'a':
    if ends("al") -> break
  case 'c':
    if ends("ance") -> break
    if ends("ence") -> break
  case 'e':
    if ends("er") -> break
  case 'i':
    if ends("ic") -> break
  case 'l':
    if ends("able") -> break
    if ends("ible") -> break
  ... more rules here...
  default: return
if m() > 1
  k = j;
```

- Precedent → preced
- Operational → operate
→ oper
-
- Interestingly
- Infuriating
- Fable → fable
- Parable → parable
- Controllable →

<http://www.eecis.udel.edu/~trnka/CIS889-11S/lectures/dan-porters.pdf>

control

Step 6

Removes a final -e

```
if b[k] == 'e'  
    int a = m()  
    if a > 1 || a == 1 && !cvc(k-1)  
        k--  
if b[k]=='l' && doublec(k) && m() > 1  
    k--
```

- Parable → parabl
- Fate → fate (cvc)
- Deflate → deflat
- Bee → bee
- Controllable →
 controll → control
- Petrol → petrol
- Stall → stall
- Resell → resel

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>

User Web Queries

Types of Queries

A user query represents the user's information needs, which is in one of the following forms:

Keyword Queries

Boolean Queries

Phrase Queries

Proximity Queries

Full Document Queries

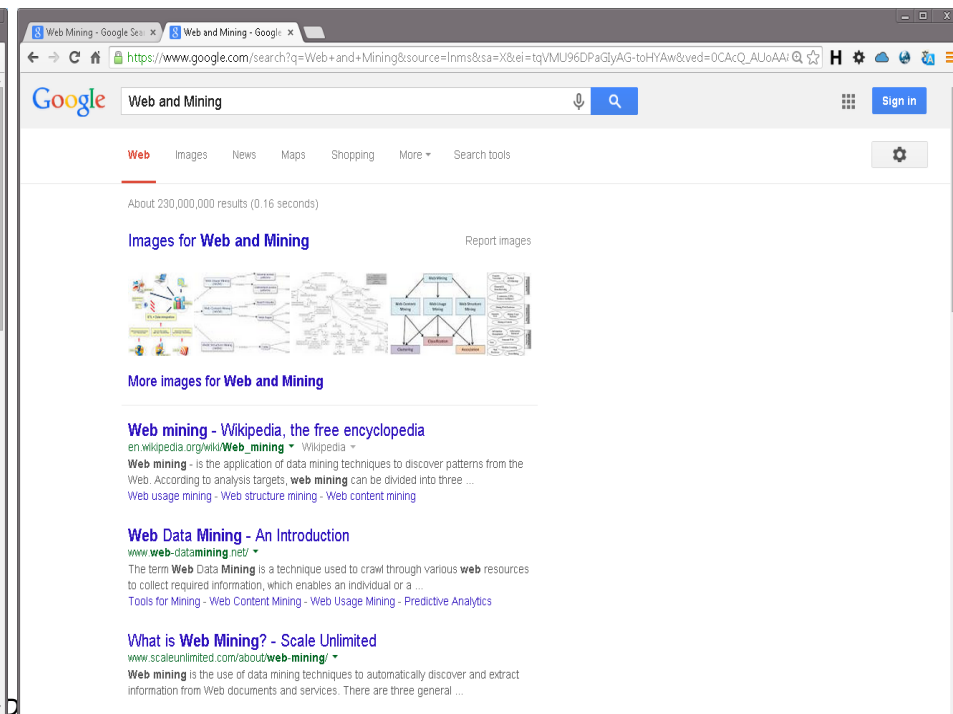
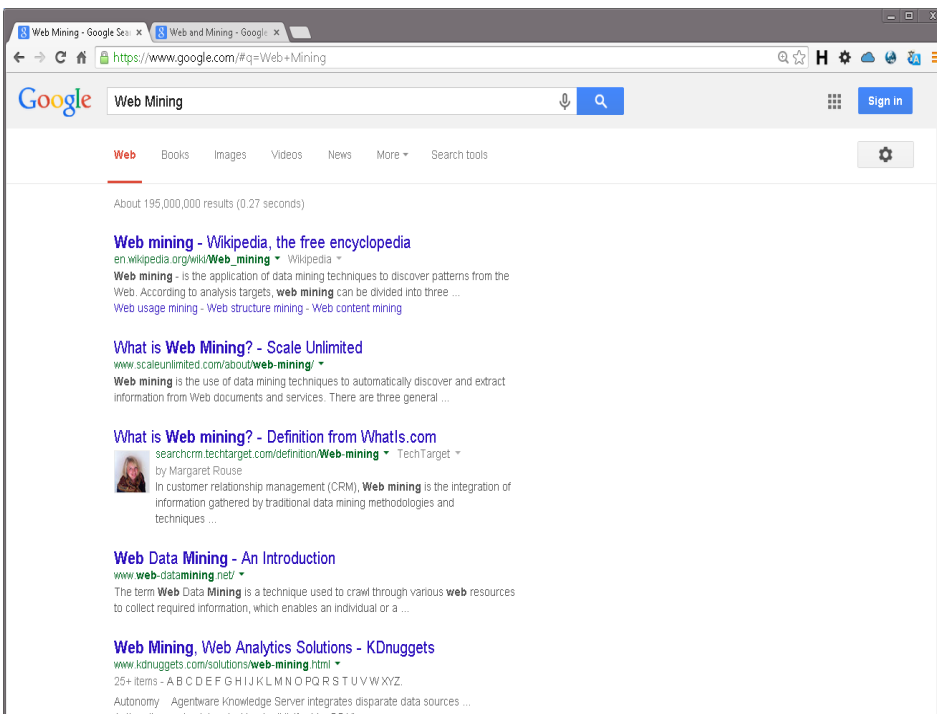
Natural Language Questions

Source: *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data* by Bing Liu



Keyword Queries

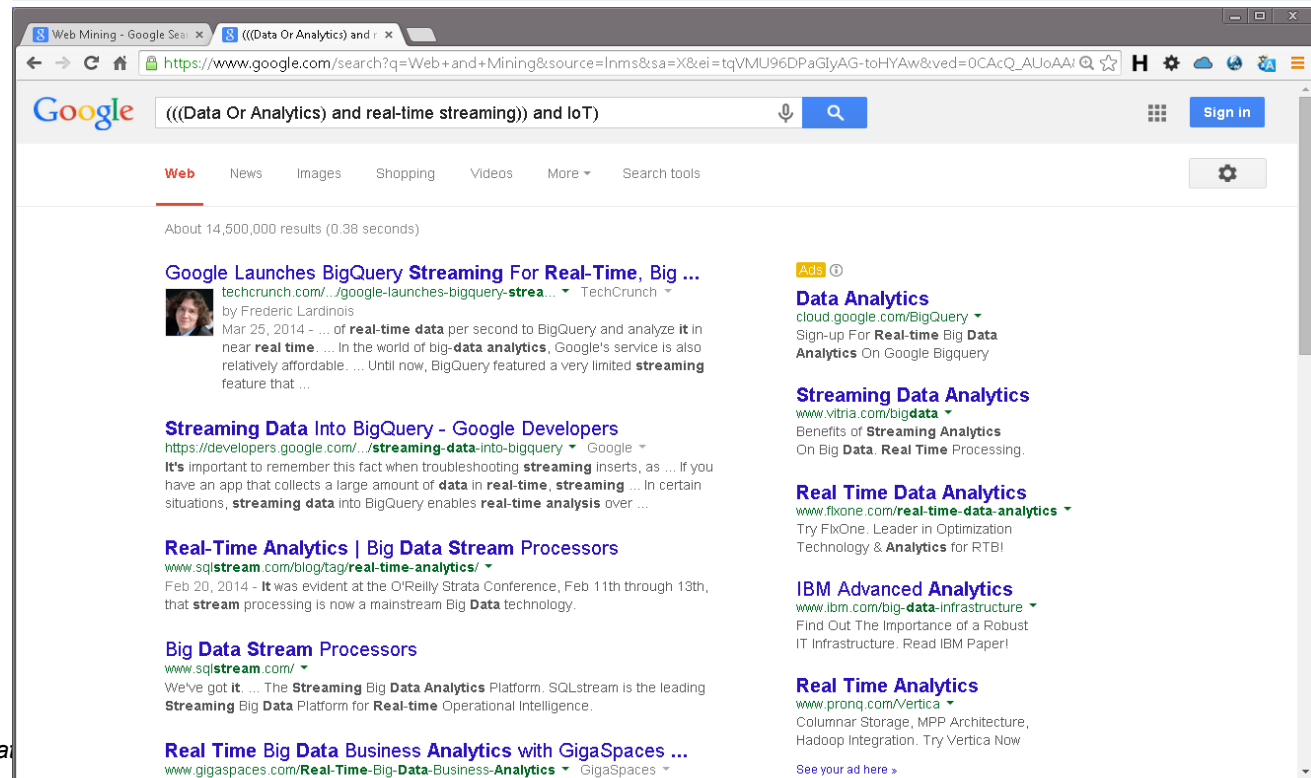
The user expresses his/her information needs with a list of (at least one) keywords (or **terms**) aiming to find documents that contain some (at least one) or all the query terms. The terms in the list are assumed to be connected **with a “soft” version of the logical AND**. For example, if one is interested in finding information about Web mining, one may issue the query ‘Web mining’ to an IR or search engine system. ‘Web mining’ is retreated as ‘Web AND mining’. The retrieval system then finds those likely relevant documents and ranks them suitably to present to the user.



Boolean Queries

The user can use Boolean operators, AND, OR, and NOT to construct complex queries. Thus, such queries consist of terms and Boolean operators. For example, 'data OR Web' is a Boolean query, which requests documents that contain the word 'data' or 'Web'. A page is returned for a Boolean query if the query is logically true in the page (i.e., **exact match**). Although one can write complex Boolean queries using the three operators, users seldom write such queries. Search engines usually support a restricted version of Boolean queries.

((Data Or Analytics) and real-time streaming)) and IoT)



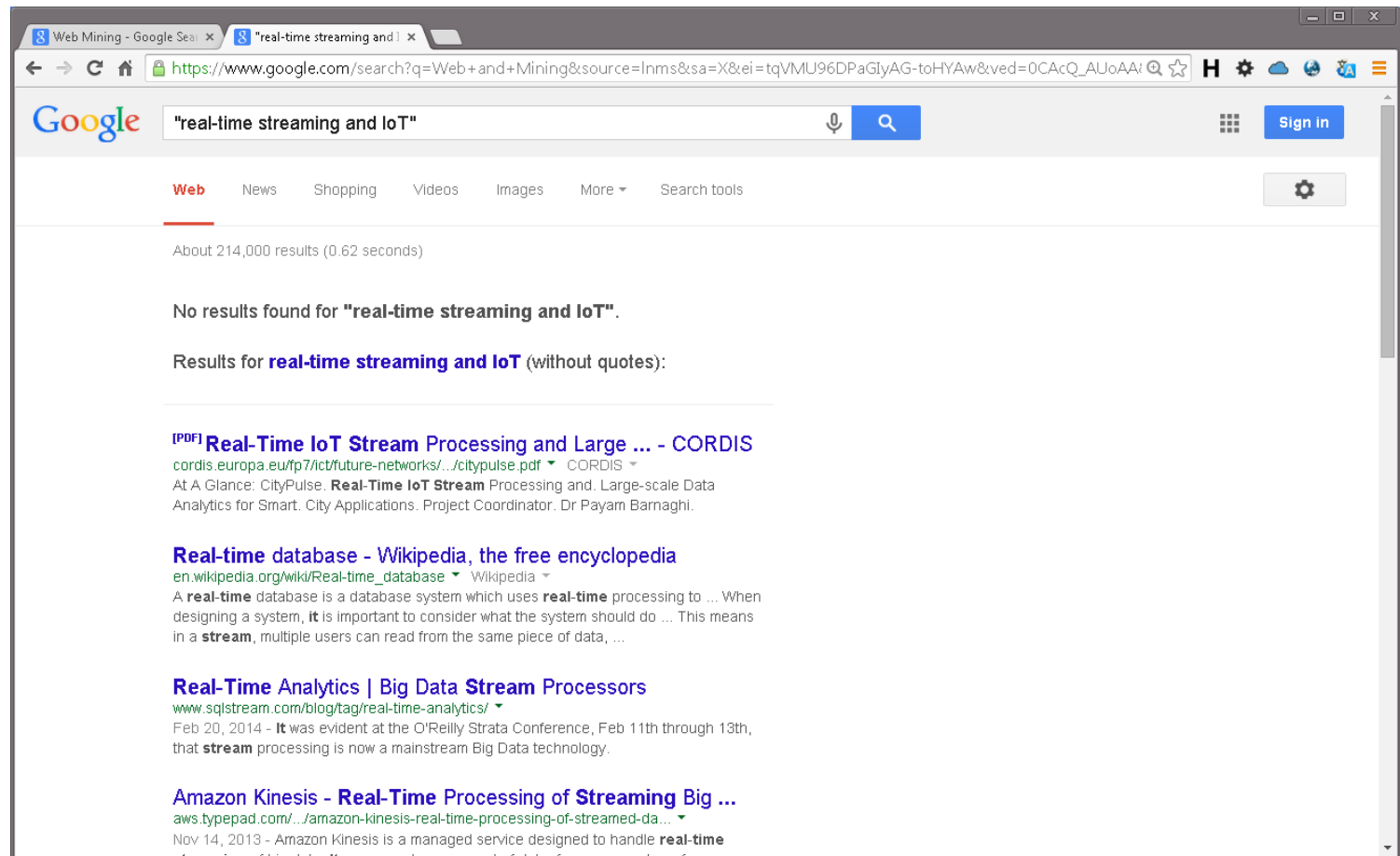
Source: Web Data



Phrase Queries

Such a query consists of a sequence of words that makes up a phrase. Each returned document must contain at least one instance of the phrase. In a search engine, a phrase query is normally enclosed with double quotes. For example, one can issue the following phrase query (including the double quotes), “Web mining techniques and applications” to find documents that contain the exact phrase.

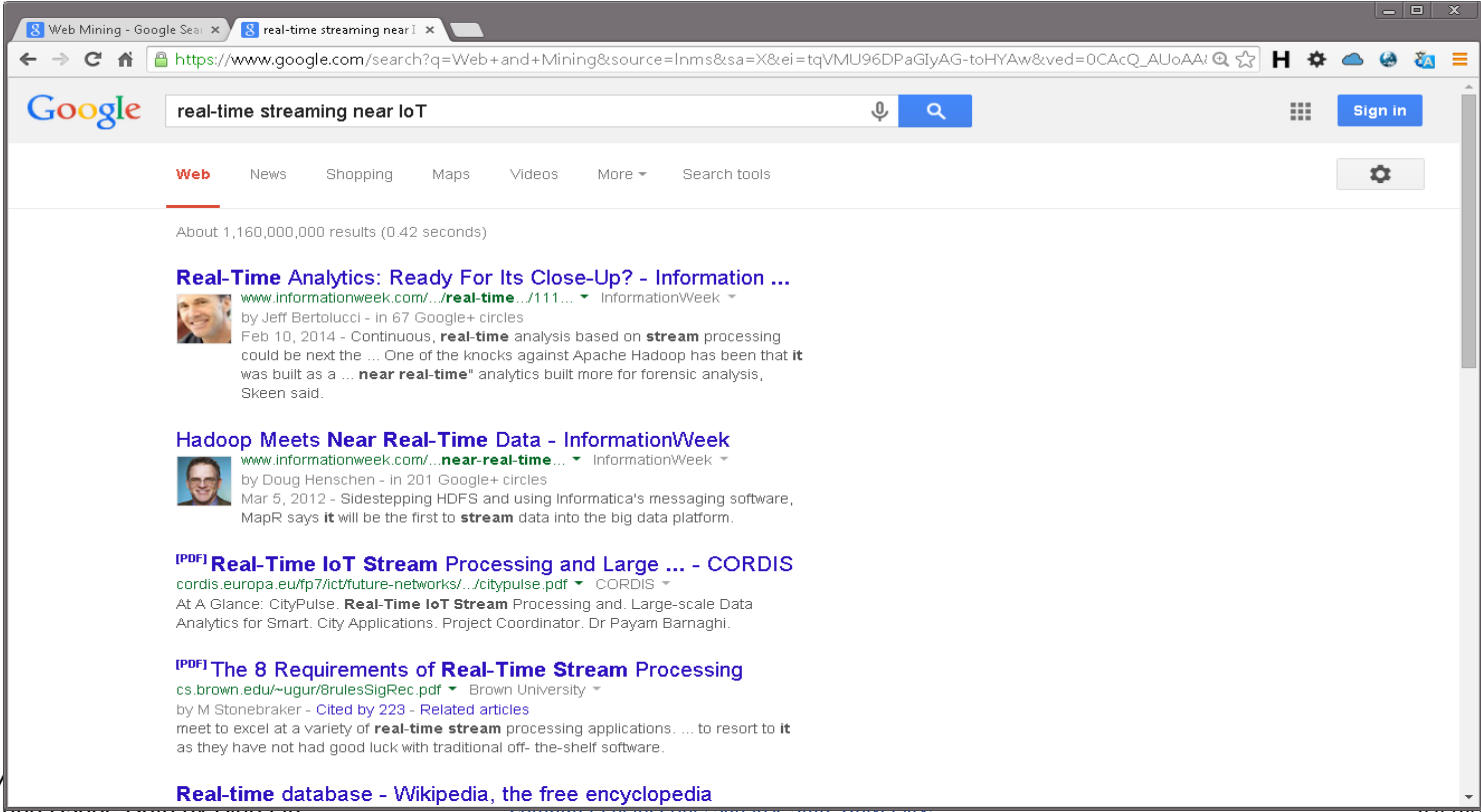
"real-time
streaming
and IoT"



Proximity Queries

The proximity query is a **relaxed version of the phrase query and can be a combination of terms and phrases**. Proximity queries seek the query terms within close proximity to each other. The closeness is used as a factor in ranking the returned documents or pages. For example, a document that contains all query terms close together is considered more relevant than a page in which the query terms are far apart. Some systems allow the user to specify the maximum allowed distance between the query terms. Most search engines consider both term proximity and term ordering in retrieval.

real-time streaming near IoT



The screenshot shows a Google search results page for the query "real-time streaming near IoT". The search bar at the top contains the query, and the results are displayed below. The first result is "Real-Time Analytics: Ready For Its Close-Up? - InformationWeek" by Jeff Bertolucci, dated Feb 10, 2014. The second result is "Hadoop Meets Near Real-Time Data - InformationWeek" by Doug Henschen, dated Mar 5, 2012. The third result is "[PDF] Real-Time IoT Stream Processing and Large ... - CORDIS" by Cordis, dated Mar 5, 2012. The fourth result is "[PDF] The 8 Requirements of Real-Time Stream Processing" by M. Stonebraker, dated Mar 5, 2012. The fifth result is "Real-time database - Wikipedia, the free encyclopedia".

Web Mining - Google Search | real-time streaming near IoT

https://www.google.com/search?q=Web+and+Mining&source=lnms&sa=X&ei=tqVMU96DPaGlyAG-toHYAw&ved=0CAcQAUoAAI

Google real-time streaming near IoT

Web News Shopping Maps Videos More Search tools

About 1,160,000,000 results (0.42 seconds)

Real-Time Analytics: Ready For Its Close-Up? - InformationWeek
www.informationweek.com/.../real-time.../111... InformationWeek
by Jeff Bertolucci - in 67 Google+ circles
Feb 10, 2014 - Continuous, **real-time** analysis based on **stream** processing could be next the ... One of the knocks against Apache Hadoop has been that it was built as a ... **near real-time** analytics built more for forensic analysis, Skeen said.

Hadoop Meets Near Real-Time Data - InformationWeek
www.informationweek.com/.../near-real-time... InformationWeek
by Doug Henschen - in 201 Google+ circles
Mar 5, 2012 - Sidestepping HDFS and using Informatica's messaging software, MapR says it will be the first to **stream** data into the big data platform.

[PDF] Real-Time IoT Stream Processing and Large ... - CORDIS
cordis.europa.eu/fp7/ict/future-networks/.../citypulse.pdf CORDIS
At A Glance: CityPulse. **Real-Time IoT Stream** Processing and. Large-scale Data Analytics for Smart. City Applications. Project Coordinator. Dr Payam Barnaghi.

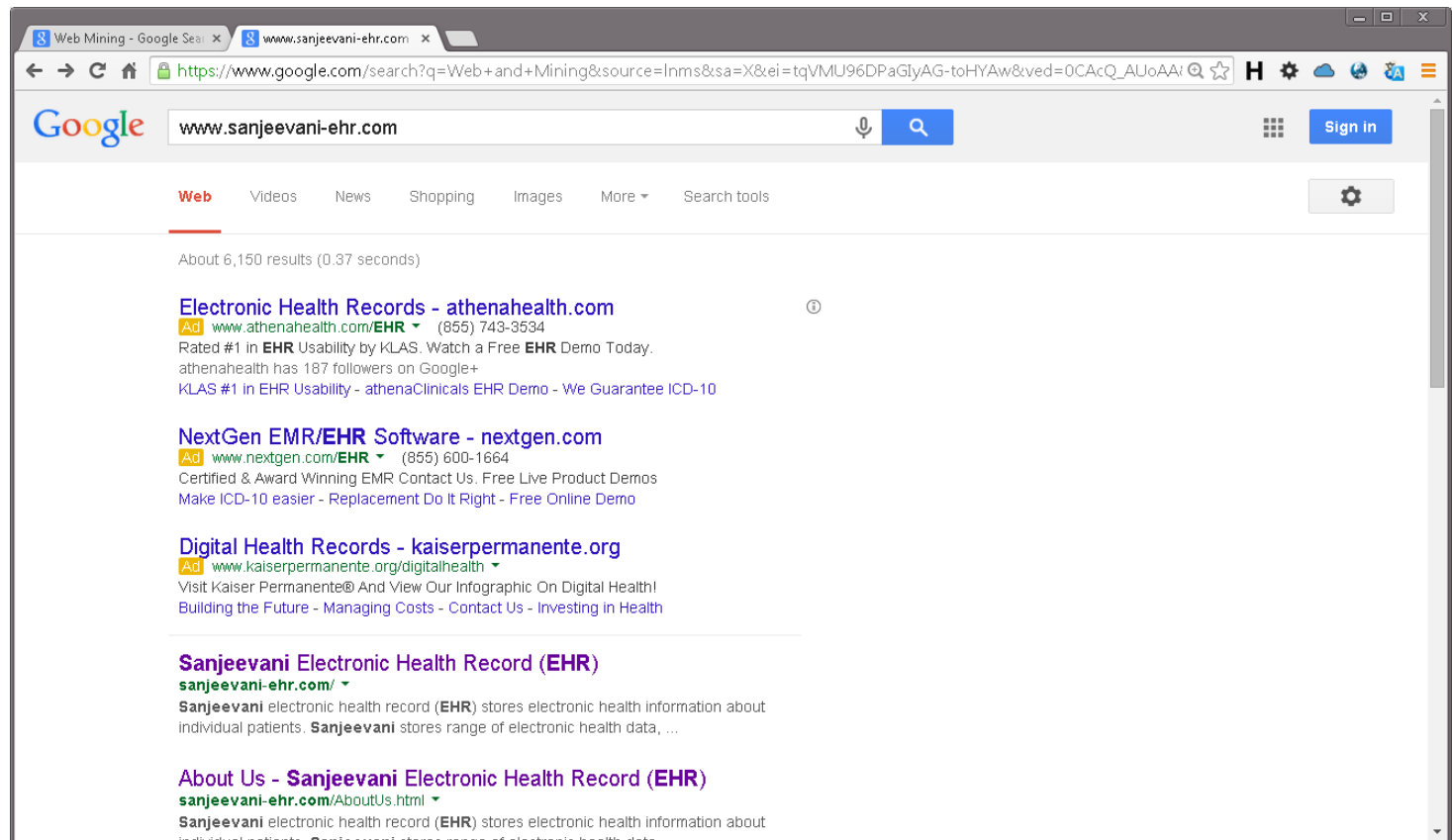
[PDF] The 8 Requirements of Real-Time Stream Processing
cs.brown.edu/~ugur/BrulesSigRec.pdf Brown University
by M Stonebraker - Cited by 223 - Related articles
meet to excel at a variety of **real-time stream** processing applications. ... to resort to it as they have not had good luck with traditional off-the-shelf software.

Real-time database - Wikipedia, the free encyclopedia



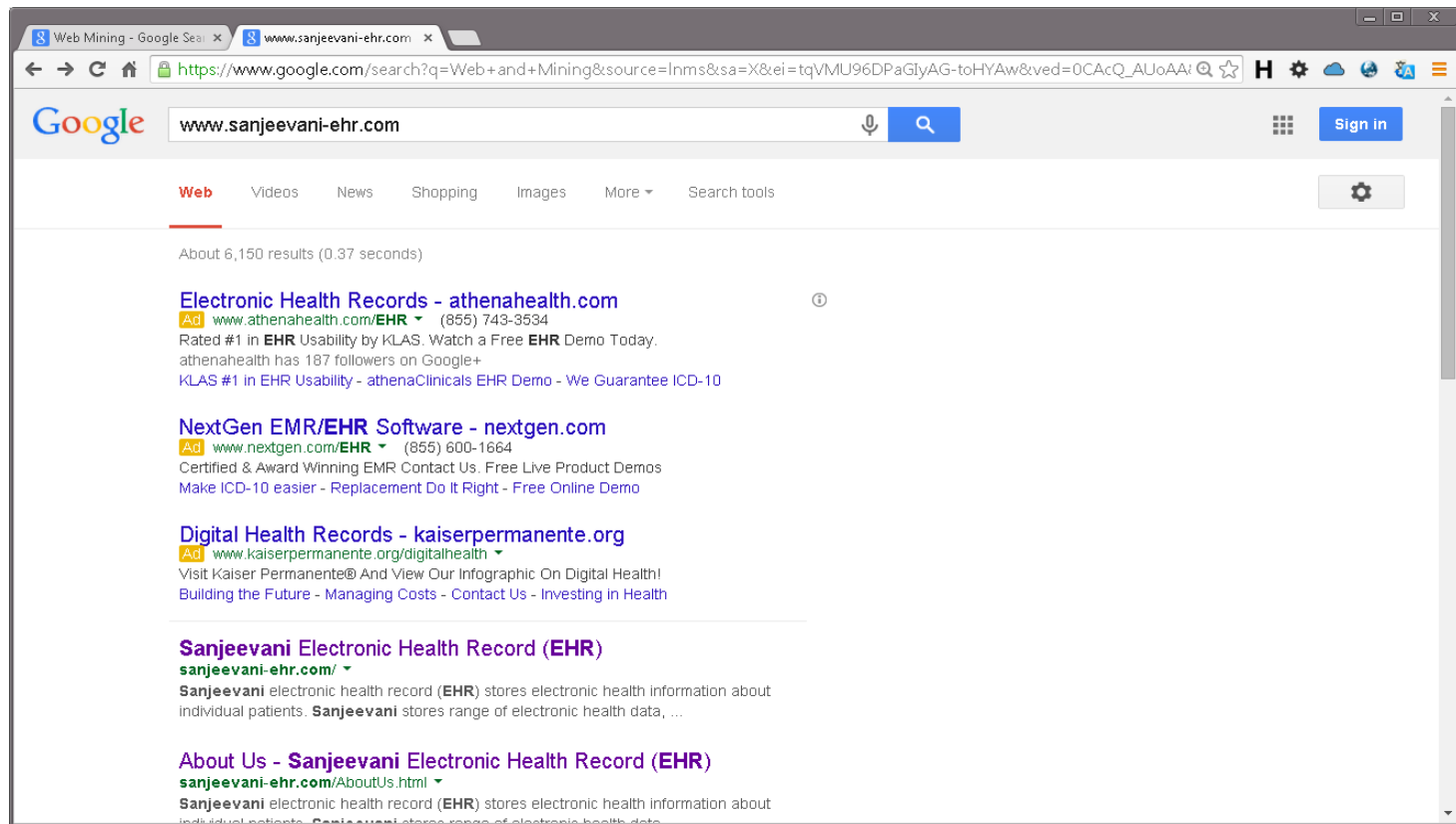
Full document Queries

When the query is a full document, the user wants to find other documents that are similar to the query document. Some search engines (e.g., Google) allow the user to issue such a query by providing the URL of a query page. Additionally, in the returned results of a search engine, each snippet may have a link called “more like this” or “similar pages.” When the user clicks on the link, a set of pages similar to the page in the snippet is returned.



Natural language Questions

This is the most complex case, and also the ideal case. The user expresses his/her information need as a natural language question. The system then finds the answer. However, such queries are still hard to handle due to the difficulty of natural language understanding. Nevertheless, this is an active research area, called **question answering**. Some search systems are starting to provide question answering services on some specific types of questions, e.g., definition questions, which ask for definitions of technical terms.



Information Retrieval Models

Information Retrieval Models

An IR model governs how a document and a query are represented and how the relevance of a document to a user query is defined. There are four main IR models:

- Boolean model,
- vector space model,
- language model and
- probabilistic model.

The most commonly used models in IR systems and on the Web are the first three models.

Given a collection of documents D , let $V = \{t_1, t_2, \dots, t_{|V|}\}$ be the set of distinctive terms in the collection, where t_i is a term. The set V is usually called the **vocabulary** of the collection, and $|V|$ is its size, i.e., the number of terms in V . A weight $w_{ij} > 0$ is associated with each term t_i of a document $\mathbf{d}_j$. For a term that does $\mathbf{d}_j \in D$, in document $\mathbf{d}_j$, $w_{ij} = 0$.

Each document $\mathbf{d}_j$ is thus represented with a term vector, $\mathbf{d}_j = (w_{1j}, w_{2j}, \dots, w_{|V|j})$,

where each weight w_{ij} corresponds to the term and $t_i \in V$,ifies the level of importance of t_i in document $\mathbf{d}_j$. The sequence of the components (or terms) in the vector is not significant.



Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf

Boolean Model

The Boolean model is the simplest to implement. A document is represented as a set of keywords. Queries are Boolean expressions of keywords, connected by AND, OR, and NOT, including the use of brackets to indicate the scope of these operators.

For example, the query “all the hotels in Rio Brazil or Hilo Hawaii, but not Hilton” is typed by the user as:
[[Rio & Brazil] | [Hilo & Hawaii]] & hotel & !Hilton]

Document Representation: In the Boolean model, documents and queries are represented as sets of terms. That is, each term is only considered present or absent in a document. Using the vector representation of the document above, the weight of term t_i in $\mathbf{d}_j$ $w_{ij} (\in \{0, 1\})$ i appears in document $\mathbf{d}_j$, and 0 otherwise, i.e.,

$$w_{ij} = \begin{cases} 1 & \text{if } t_i \text{ appears in } \mathbf{d}_j \\ 0 & \text{otherwise.} \end{cases}$$

Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf



Boolean Model

Boolean Queries: query terms are combined logically using the Boolean operators **AND**, **OR**, and **NOT**, which have their usual semantics in logic. Thus, a Boolean query has a precise semantics. For instance, the query, $((x \text{ AND } y) \text{ AND } (\text{NOT } z))$ says that a retrieved document must contain both the terms x and y but not z . As another example, the query expression $(x \text{ OR } y)$ means that at least one of these terms must be in each retrieved document. Here, we assume that x , y and z are terms. In general, they can be Boolean expressions themselves.

Document Retrieval: Given a Boolean query, the system retrieves every document that makes the query logically true. Thus, the retrieval is based on the binary decision criterion, i.e., a document is either relevant or irrelevant. Intuitively, this is called **exact match**. There is no notion of partial match or ranking of the retrieved documents. This is one of the major disadvantages of the Boolean model, which often leads to poor retrieval results. It is quite clear that the frequency of terms and their proximity contribute significantly to the relevance of a document.



Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf

Vector Space Model

The vector space model of information retrieval is a very successful statistical method proposed by Salton (1989). It generates **weighted term vectors** for each document in the collection, and for the user query. **Then the retrieval is based on the similarity between the query vector and document vectors.** The output documents are ranked according to this similarity. The similarity is based on the occurrence frequencies of the keywords in the query and in the documents.

Documentation Representation:

A document in the vector space model is represented as a weight vector, in which each component weight is computed based on some variation of TF or TF-IDF scheme. The weight w_{ij} of term t_i in document d_j is no longer in $\{0, 1\}$ as in the Boolean model, but can be any number.

Term Frequency (TF) Scheme: In this method, the weight of a term t_i in document d_j is the number of times that t_i appears in document d_j , denoted by f_{ij} . Normalization may also be applied. The shortcoming of the TF **scheme is that it does not consider the situation where a term appears in many documents of the collection. Such a term may not be discriminative.**

TF-IDF Scheme: This is the most well known weighting scheme, where TF still stands for the **term frequency** and IDF the **inverse document frequency**. There are several variations of this scheme. Here we only give the most basic one.



Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf



Vector Space Model

- ▶ The most well known weighting scheme
 - ▶ TF: still **term frequency**
 - ▶ IDF: **inverse document frequency**.

N : total number of docs

df_i : the number of docs that t_i appears.

- ▶ The final TF-IDF term weight is:

$$w_{ij} = tf_{ij} \times idf_i.$$

$$tf_{ij} = \frac{f_{ij}}{\max\{f_{1j}, f_{2j}, \dots, f_{|V|j}\}}$$

$$idf_i = \log \frac{N}{df_i}$$

Source: http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf



Vector Space Model

Retrieval in vector space model

- ▶ **Query $\mathbf{q}$** is represented in the same way or slightly differently.
- ▶ **Relevance of $\mathbf{d}_j$ to $\mathbf{q}$** : Compare the similarity of query $\mathbf{q}$ and document $\mathbf{d}_j$.
- ▶ Cosine similarity (the cosine of the angle between the two vectors)

$$\text{cosine}(\mathbf{d}_j, \mathbf{q}) = \frac{\langle \mathbf{d}_j \bullet \mathbf{q} \rangle}{\|\mathbf{d}_j\| \times \|\mathbf{q}\|} = \frac{\sum_{i=1}^{|V|} w_{ij} \times w_{iq}}{\sqrt{\sum_{i=1}^{|V|} w_{ij}^2} \times \sqrt{\sum_{i=1}^{|V|} w_{iq}^2}}$$

- ▶ Cosine is also commonly used in text clustering

Source: Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data by Bing Liu



Vector Space Model

Ranking

- ▶ The similarity values can then be used to rank the documents



Source: *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data* by Bing Liu

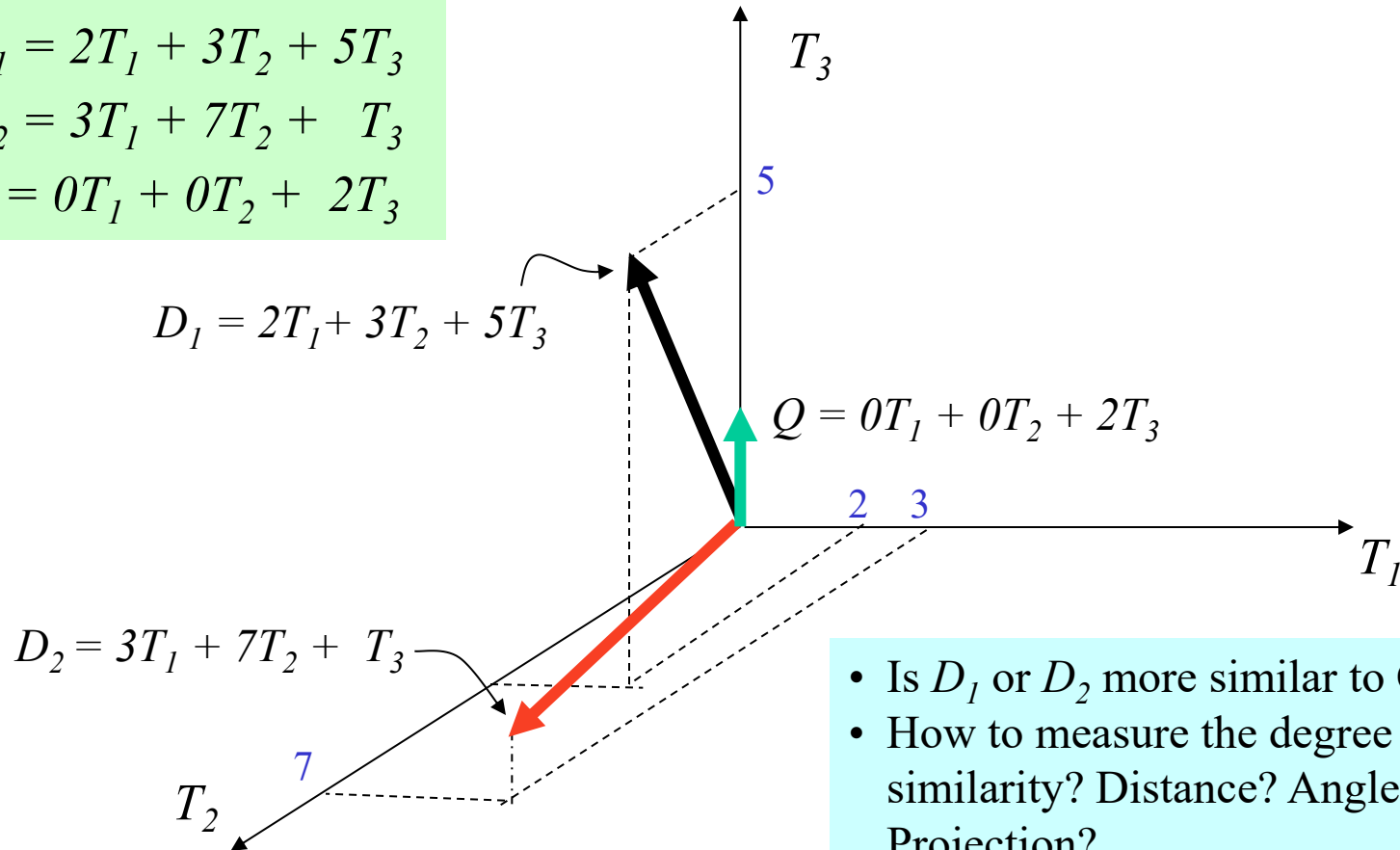
Graphic Representation

Example:

$$D_1 = 2T_1 + 3T_2 + 5T_3$$

$$D_2 = 3T_1 + 7T_2 + T_3$$

$$Q = 0T_1 + 0T_2 + 2T_3$$

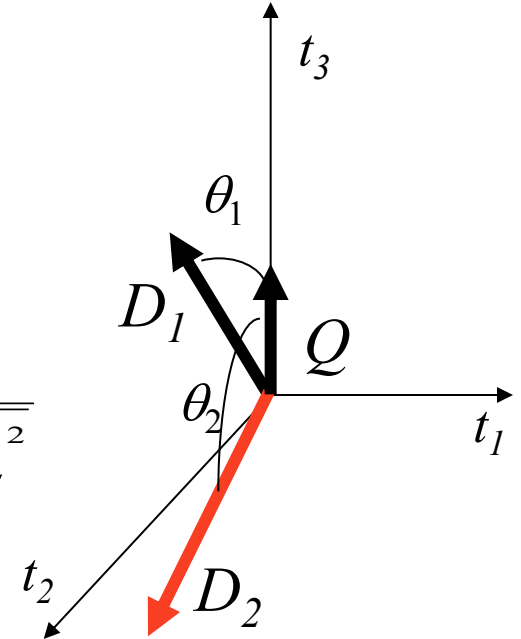


- Is D_1 or D_2 more similar to Q ?
- How to measure the degree of similarity? Distance? Angle? Projection?

Cosine Similarity Measure

- Cosine similarity measures the cosine of the angle between two vectors.
- Inner product normalized by the vector lengths.

$$\text{CosSim}(\vec{d}_j, \vec{q}) = \frac{\vec{d}_j \cdot \vec{q}}{|\vec{d}_j| \cdot |\vec{q}|} = \frac{\sum_{i=1}^t (w_{ij} \cdot w_{iq})}{\sqrt{\sum_{i=1}^t w_{ij}^2 \cdot \sum_{i=1}^t w_{iq}^2}}$$



$$\begin{aligned} D_1 &= 2T_1 + 3T_2 + 5T_3 & \text{CosSim}(D_1, Q) &= 10 / \sqrt{(4+9+25)(0+0+4)} = 0.81 \\ D_2 &= 3T_1 + 7T_2 + 1T_3 & \text{CosSim}(D_2, Q) &= 2 / \sqrt{(9+49+1)(0+0+4)} = 0.13 \\ Q &= 0T_1 + 0T_2 + 2T_3 \end{aligned}$$

D_1 is 6 times better than D_2 using cosine similarity but only 5 times better using inner product.

Example

- ▶ A document space is defined by three terms:
 - ▶ $V = \{\text{hardware, software, users}\}$
- ▶ A set of documents are defined as:
 - ▶ $A1=(1, 0, 0), \quad A2=(0, 1, 0), \quad A3=(0, 0, 1)$
 - ▶ $A4=(1, 1, 0), \quad A5=(1, 0, 1), \quad A6=(0, 1, 1)$
 - ▶ $A7=(1, 1, 1) \quad A8=(1, 0, 1). \quad A9=(0, 1, 1)$
- ▶ If the Query is “hardware and software” what documents should be retrieved?
 - ▶ Boolean query matching
 - ▶ Vector space matching

$(1,1,0)$

Source: *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data* by Bing Liu



Example

- ▶ In Boolean query matching:
 - ▶ document A4,A7 will be retrieved (“AND”)
- ▶ In similarity matching (cosine):
 - ▶ $q=(1, 1, 0)$
 - ▶ $S(q, A1)=0.71$
 - ▶ $S(q, A2)=0.71$
 - ▶ $S(q, A3)=0$
 - ▶ $S(q, A4)=1$
 - ▶ $S(q, A5)=0.5$
 - ▶ $S(q, A6)=0.5$
 - ▶ $S(q, A7)=0.82$
 - ▶ $S(q, A8)=0.5$
 - ▶ $S(q, A9)=0.5$
 - ▶ Document retrieved set (with ranking)=
 - ▶ $\{A4, A7, A1, A2, A5, A6, A8, A9\}$



Source: *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data* by Bing Liu

Example (Deerwester)

Query Vector (Human computer interface)

<http://ttic.uchicago.edu/~dmcalister/course/lec8.ppt>

Titles

- c1: Human machine interface for Lab ABC computer applications
- c2: A survey of user opinion of computer system response time
- c3: The EPS user interface management system
- c4: System and human system engineering testing of EPS
- c5: Relation of user-perceived response time to error measurement
- m1: The generation of random, binary, unordered trees
- m2: The intersection graph of paths in trees
- m3: Graph minors IV: Widths of trees and well-quasi-ordering
- m4: Graph minors: A survey

Terms

Documents

	c1	c2	c3	c4	c5	m1	m2	m3	m4
human	1	0	0	1	0	0	0	0	0
interface	1	0	1	0	0	0	0	0	0
computer	1	1	0	0	0	0	0	0	0
user	0	1	1	0	1	0	0	0	0
system	0	1	1	2	0	0	0	0	0
response	0	1	0	0	1	0	0	0	0
time	0	1	0	0	1	0	0	0	0
EPS	0	0	1	1	0	0	0	0	0
survey	0	1	0	0	0	0	0	0	1
trees	0	0	0	0	0	1	1	1	0
graph	0	0	0	0	0	0	1	1	1
minors	0	0	0	0	0	0	0	1	1

2-D Plot of Terms and Docs from Example

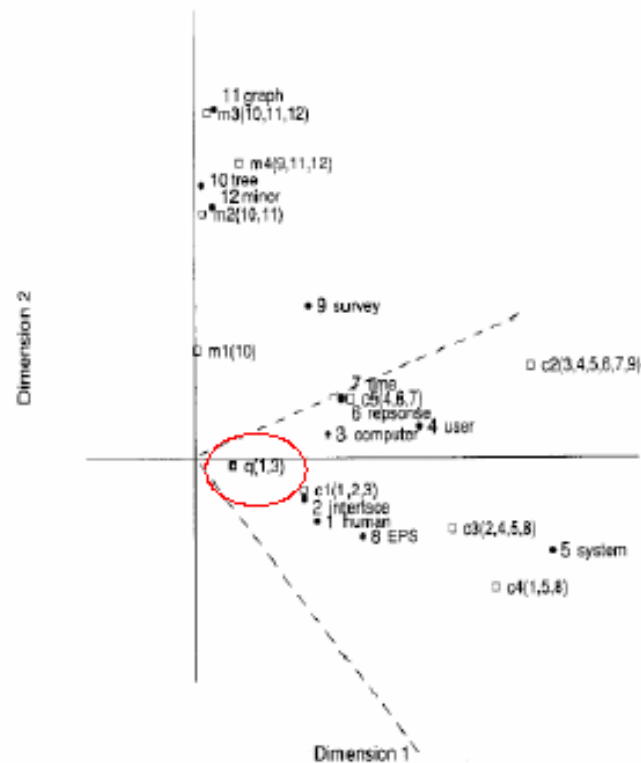


FIG. 1. A two-dimensional plot of 12 Terms and 9 Documents from the sample TM set. Terms are represented by filled circles. Documents are shown as open squares, and component terms are indicated parenthetically. The query ("human computer interaction") is represented as a pseudo document at point q . Axes are scaled for Document-Document or Term-Term comparisons. The dotted cone represents the region whose points are within a cosine of .9 from the query q . All documents about human-computer (c1-c5) are "near" the query (i.e., within this cone), but none of the graph theory documents (m1-m4) are nearby. In this reduced space, even documents c3 and c5 which share no terms with the query are near it.

In Class Exercise (Graded)

Given the following documents and queries:

D1: You say goodbye, I say hello

D2: You say stop, I say go

D3: Hello, hello, you say goodbye

D4: I say yes, you say no

Q1: say goodbye

Q2: you hello

- Is D_1, D_2, D_3 , or D_4 more similar to Q_1 ?
- How to measure the degree of similarity? Distance? Angle? Projection?

- Is D_1, D_2, D_3 , or D_4 more similar to Q_2 ?
- How to measure the degree of similarity? Distance? Angle? Projection?



Next Class

- Invested Index
- LSI
- IR Evaluation Method
- Web Search
- Meta Search



Source: *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data* by Bing Liu

References

- <http://www.slideshare.net/mounialalmas/introduction-to-information-retrieval-models>
- Language Models for Information Retrieval, <http://nlp.stanford.edu/IR-book/pdf/12lmodel.pdf>
- Information Retrieval Models, http://www.ccs.neu.edu/home/ekanou/ISU535.09X2/Handouts/Review_Material/irmodels.pdf
- Introduction to Information Retrieval, <http://www.deri.ie/fileadmin/documents/teaching/tutorials/tutorial-ColmOriordan.pdf>
- Information Retrieval on the Internet, http://www.site.uottawa.ca/~diana/csi4107/IR_draft.pdf
- IR, <http://www.ccs.neu.edu/course/cs6200f13/cs6200-f13-2.pdf>
- The Anatomy of a Large-Scale Hypertextual Web Search Engine, <http://infolab.stanford.edu/~backrub/google.html>

